

FIAP TECH CHALLENGE 1 - GRUPO SSP

2. ANÁLISE DOS DADOS

2.1. Estabelecendo a variável `BASE_PATH`

```
In [1]: import pandas as pd
        from pathlib import Path

        # Considerando que o Notebook está sendo rodado em /src/notebooks/
        BASE_PATH = Path.cwd().resolve().parent.parent
        print(f"📁 Base path: {BASE_PATH.absolute()}")
```

📁 Base path: /Users/matias/Projetos/fiap/tech-challenge-1

2.2. Gerando um Dataframe único com todos os arquivos parquet concatenados

```
In [2]: # Carregar todos os arquivos parquet do diretório
        parquet_dir = BASE_PATH / "data" / "raw" / "VIOLBR24.parquet"

        # Lista todos os arquivos .parquet no diretório
        parquet_files = list(parquet_dir.glob("*.parquet"))
        print(f"📁 Encontrados {len(parquet_files)} arquivos parquet")

        # Carrega cada arquivo parquet e combina em um único dataframe
        dfs = []
        for parquet_file in parquet_files:
            df = pd.read_parquet(parquet_file)
            dfs.append(df)
            # print(f"✓ Carregado: {parquet_file.name} ({len(df)} linhas)")

        # Concatena todos os dataframes
        df = pd.concat(dfs, ignore_index=True)
        print(f"\n✅ DataFrame final criado com {len(df)} linhas e {len(df.columns)}")
```

📁 Encontrados 23 arquivos parquet

✅ DataFrame final criado com 662803 linhas e 160 colunas

2.3. Aplicando filtros para delimitar escopo

Filtros Aplicados:

- Sexo Feminino
- Lesões que não sejam autoprovacadas

```
In [3]: filter = (  
        (df['CS_SEX0'] == 'F') &  
        (df['LES_AUTOP'] == '2')  
    )  
    filtered_df = df[filter].copy()
```

```
In [4]: # Visualização inicial do dataframe  
print("📄 Informações do DataFrame:")  
print(f"Shape: {filtered_df.shape}")  
print(f"\nColunas ({len(filtered_df.columns)}):")  
print(filtered_df.columns.tolist())  
print(f"\nTipos de dados:")  
print(filtered_df.dtypes)  
print(f"\nPrimeiras linhas:")  
filtered_df.head()
```

📄 Informações do DataFrame:

Shape: (294393, 160)

Colunas (160):

```
['TP_NOT', 'ID_AGRAVO', 'DT_NOTIFIC', 'SEM_NOT', 'NU_ANO', 'SG_UF_NOT', 'ID_MUNICIP', 'ID_UNIDADE', 'DT_OCOR', 'SEM_PRI', 'ANO_NASC', 'NU_IDADE_N', 'CS_SEXO', 'CS_GESTANT', 'CS_RACA', 'CS_ESCOL_N', 'SG_UF', 'ID_MN_RESI', 'ID_PAIS', 'NDUPLIC', 'DT_INVEST', 'ID_OCUPA_N', 'SIT_CONJUG', 'DEF_TRANS', 'DEF_FISICA', 'DEF_MENTAL', 'DEF_VISUAL', 'DEF_AUDITI', 'TRAN_MENT', 'TRAN_COMP', 'DEF_OUT', 'DEF_ESPEC', 'SG_UF_OCOR', 'ID_MN_OCOR', 'HORA_OCOR', 'LOCAL_OCOR', 'LOCAL_ESPE', 'OUT_VEZES', 'LES_AUTOP', 'VIOL_FISIC', 'VIOL_PSICO', 'VIOL_TORT', 'VIOL_SEXU', 'VIOL_TRAF', 'VIOL_FINAN', 'VIOL_NEGLI', 'VIOL_INFAN', 'VIOL_LEGAL', 'VIOL_OUTR', 'VIOL_ESPEC', 'AG_FORCA', 'AG_ENFOR', 'AG_OBJETO', 'AG_CORTE', 'AG_QUENTE', 'AG_ENVEN', 'AG_FOGO', 'AG_AMEACA', 'AG_OUTROS', 'AG_ESPEC', 'SEX_ASSEDI', 'SEX_ESTUPR', 'SEX_PUDOR', 'SEX_PORNO', 'SEX_EXPLO', 'SEX_OUTRO', 'SEX_ESPEC', 'PEN_ORAL', 'PEN_ANAL', 'PEN_VAGINA', 'PROC_DST', 'PROC_HIV', 'PROC_HEPB', 'PROC_SANG', 'PROC_SEMEN', 'PROC_VAGIN', 'PROC_CONTR', 'PROC_ABORT', 'CONS_ABORT', 'CONS_GRAV', 'CONS_DST', 'CONS_SUIC', 'CONS_MENT', 'CONS_COMP', 'CONS_ESTRE', 'CONS_OUTR', 'CONS_ESPEC', 'LESAO_NAT', 'LESAO_ESPE', 'LESAO_CORP', 'NUM_ENVOLV', 'REL_SEXUAL', 'REL_PAI', 'REL_MAE', 'REL_PAD', 'REL_CONJ', 'REL_EXCON', 'REL_NAMO', 'REL_EXNAM', 'REL_FILHO', 'REL_DESCO', 'REL_IRMAO', 'REL_CONHEC', 'REL_CUIDA', 'REL_PATRAO', 'REL_INST', 'REL_POL', 'REL_PROPRI', 'REL_OUTROS', 'REL_ESPEC', 'AUTOR_SEXO', 'AUTOR_ALCO', 'ENC_SAUDE', 'ENC_TUTELA', 'ENC_VARA', 'ENC_ABRIGO', 'ENC_SENTIN', 'ENC_DEAM', 'ENC_DPCA', 'ENC_DELEG', 'ENC_MPU', 'ENC_MULHER', 'ENC_CRIAS', 'ENC_IML', 'ENC_OUTR', 'ENC_ESPEC', 'REL_TRAB', 'REL_CAT', 'CIRC_LESAO', 'CLASSI_FIN', 'EVOLUCAO', 'DT_OBITO', 'DT_DIGITA', 'DT_TRANSUS', 'DT_TRANSDM', 'DT_TRANSSM', 'DT_TRANSRM', 'DT_TRANSRS', 'DT_TRANSSE', 'REL_MAD', 'TPUNINOT', 'ORIENT_SEX', 'IDENT_GEN', 'VIOL_MOTIV', 'CICL_VID', 'REDE_SAU', 'ASSIST_SOC', 'REDE_EDUCA', 'ATEND_MULH', 'CONS_TUTEL', 'CONS_IDO', 'DELEG_IDOS', 'DIR_HUMAN', 'MPU', 'DELEG_CRIA', 'DELEG_MULH', 'DELEG', 'INFAN_JUV', 'DEFEN_PUBL', 'DT_ENCERRA']
```

Tipos de dados:

```
TP_NOT      object
ID_AGRAVO   object
DT_NOTIFIC  object
SEM_NOT      object
NU_ANO      object
...
DELEG_MULH  object
DELEG       object
INFAN_JUV   object
DEFEN_PUBL  object
DT_ENCERRA  object
Length: 160, dtype: object
```

Primeiras linhas:

Out [4]:

	TP_NOT	ID_AGRAVO	DT_NOTIFIC	SEM_NOT	NU_ANO	SG_UF_NOT	ID_MUNICIP
0	2	Y09	20240610	202424	2024	29	292740
1	2	Y09	20240610	202424	2024	29	291920
4	2	Y09	20240610	202424	2024	35	353440
5	2	Y09	20240610	202424	2024	35	355030
6	2	Y09	20240610	202424	2024	41	412350

5 rows × 160 columns

```
In [5]: import json

# Carregar o dicionário de campos
dicionario_path = BASE_PATH / "src" / "data" / "dicionario_campos_sinan.json"
with open(dicionario_path, 'r', encoding='utf-8') as f:
    dicionario = json.load(f)

# Filtrar campos com relevante = True
campos_relevantes = [
    campo for campo, info in dicionario.items()
    if info.get('relevante', False) == True
]

# Filtrar apenas as colunas que existem no dataframe
colunas_existentes = [col for col in campos_relevantes if col in filtered_df]
colunas_nao_existentes = [col for col in campos_relevantes if col not in filtered_df]

if colunas_nao_existentes:
    print(f"\n⚠️ Aviso: {len(colunas_nao_existentes)} campos do dicionário não existem no dataframe")
    print(colunas_nao_existentes)

# Criar subconjunto do dataframe com apenas as colunas relevantes
df_relevante = filtered_df[colunas_existentes].copy()

print(f"\n✅ DataFrame com campos relevantes criado:")
print(f"Shape: {df_relevante.shape}")
```

✅ DataFrame com campos relevantes criado:
Shape: (294393, 79)

```
In [6]: # Valores null/NaN
valores_null = df_relevante.isna().sum()

# Strings vazias ou com apenas espaços
valores_string_vazia = df_relevante.apply(
    lambda col: (col.astype(str).str.strip() == '').sum()
)

# print(f'valores_null: {valores_null}')
# print(f'valores_string_vazia: {valores_string_vazia}')
```

```

# dataframes com valores null e valores string vazia
# valores_vazio_df = pd.concat([valores_null, valores_string_vazia], axis=1,
# print(valores_vazio_df)

# printar somente LESAO_ESPE (linhas = campos, colunas = tipo de vazio)
# print(valores_vazio_df.loc['LESAO_ESPE'])

# Total combinado
valores_vazios = valores_null + valores_string_vazia

percentual_vazios = (valores_vazios / len(df_relevante)) * 100
# formatar percentual_vazios em 2 decimais
percentual_vazios = percentual_vazios.round(5)
# ordenar percentual_vazios de forma decrescente
percentual_vazios = percentual_vazios.sort_values(ascending=False)

# print(f'valores_vazios (total): {valores_vazios}')

print(f'percentual_vazios: {percentual_vazios}')

```

```

percentual_vazios: TPUNINOT      100.00000
ENC_ESPEC      99.99864
LESAO_ESPE      99.99796
LESAO_CORP      99.99185
LESAO_NAT      99.99049
...
SG_UF      0.01155
ID_AGRAVO      0.00000
CS_SEX0      0.00000
LES_AUTOP      0.00000
DT_OCOR      0.00000
Length: 79, dtype: float64

```

```

In [7]: # criar dataframe em que as colunas são as que tem menos de 10% de vazios
cleaned_df = df_relevante.loc[:, percentual_vazios < 10]
# print(cleaned_df.shape)
# print(cleaned_df.columns)
# print(cleaned_df.head())

# recalcular percentual de vazios
ev2 = cleaned_df.apply(
    lambda col: (col.astype(str).str.strip() == '').sum()
)
pv2 = (ev2 / len(cleaned_df)) * 100
pv2 = pv2.round(2)
pv2 = pv2.sort_values(ascending=False)

print(f'percentual_vazios: {pv2}')

# print quantidade de colunas do dataframe atual
print(f'quantidade de colunas do dataframe atual: {len(cleaned_df.columns)}')

# salvar em data/processed/df_cleaned.parquet
cleaned_df.to_parquet(BASE_PATH / "data" / "processed" / "df_cleaned.parquet")

```

percentual_vazios: CS_ESCOL_N 3.36

AG_AMEACA	1.96
CONS_TUTEL	1.53
ATEND_MULH	1.52
REL_CUIDA	1.44
REL_POL	1.44
REL_PATRAO	1.44
REL_MAD	1.44
REL_INST	1.44
REL_PROPRI	1.43
REL_PAD	1.42
REL_IRMAO	1.42
REL_FILHO	1.42
REL_EXNAM	1.42
REL_TRAB	1.41
REL_OUTROS	1.40
REL_NAMO	1.40
REL_DESCO	1.39
REL_EXCON	1.36
REL_CONHEC	1.33
VIOL_OUTR	1.30
VIOL_LEGAL	1.29
VIOL_INFAN	1.29
VIOL_TRAF	1.29
VIOL_FINAN	1.28
VIOL_TORT	1.27
REL_CONJ	1.26
VIOL_PSICO	1.21
REL_PAI	1.20
SIT_CONJUG	1.18
VIOL_SEXU	1.14
REL_MAE	1.12
DEF_OUT	1.05
DEF_VISUAL	0.96
DEF_AUDITI	0.96
DEF_MENTAL	0.95
DEF_FISICA	0.94
IDENT_GEN	0.93
REDE_SAU	0.87
ANO_NASC	0.71
VIOL_FISIC	0.67
VIOL_NEGLI	0.67
VIOL_MOTIV	0.53
DEF_TRANS	0.53
ORIENT_SEX	0.42
OUT_VEZES	0.33
DT_ENCERRA	0.32
CS_RACA	0.19
AUTOR_SEXO	0.15
SG_UF	0.01
ID_AGRAVO	0.00
LES_AUTOP	0.00
CS_SEXO	0.00
DT_OCOR	0.00

dtype: float64

quantidade de colunas do dataframe atual: 54

Distribuição de Valores

LEGENDA: Para a maioria das respostas:

1. Sim
2. Não
3. Ignorado

Orientação Sexual:

1. Heterossexual
2. Homossexual (gay/lésbica)
3. Bissexual
4. Não se aplica
5. Ignorado

Autor_Sexo:

1. Masculino
2. Feminino
3. Ambos os sexos 9 .Ignorado

Identidade de Gênero:

1. Travesti
2. Transexual Mulher
3. Transexual Homem
4. Não se aplica
5. Ignorado

Motivo da Violência:

1. Sexismo
2. Homofobia/Lesbofobia Bifobia/Transfobia
3. Racismo
4. Intolerância religiosa
5. Xenofobia
6. Conflito geracional
7. Situação de rua
8. Deficiência
9. Outros
10. Não se aplica
11. Ignorado

Escolaridade:

1. 1ª a 4ª série incompleta do EF
2. 4ª série completa do EF (antigo 1º grau)
3. 5ª à 8ª série incompleta do EF (antigo ginásio ou 1º grau)
4. Ensino fundamental completo (antigo ginásio ou 1º grau)
5. Ensino médio incompleto (antigo colegial ou 2º grau)
6. Ensino médio completo (antigo colegial ou 2º grau)
7. Educação superior incompleta
8. Educação superior completa
9. Ignorado
10. Não se aplica

Raça: 1- branca 2- preta 3- amarela 4- parda 5- indígena 9 Ignorado

Deficiências:

1. Sim
2. Não
3. Não se aplica
4. Ignorado

```
In [8]: # Analisar a distribuição de valores de cada coluna
for col in cleaned_df.columns:
    print(f'Distribuição de valores da coluna {col}:')
    print(cleaned_df[col].value_counts())
    print('\n')
```


Distribuição de valores da coluna AG_AMEACA:

AG_AMEACA

2 213212

1 68460

9 6958

5763

Name: count, dtype: int64

Distribuição de valores da coluna ANO_NASC:

ANO_NASC

2010 9363

2011 8322

2009 7915

1999 6645

2000 6458

...

1914 2

1921 2

1916 1

1911 1

1919 1

Name: count, Length: 113, dtype: int64

Distribuição de valores da coluna ATEND_MULH:

ATEND_MULH

2 239319

1 40208

9 10394

4472

Name: count, dtype: int64

Distribuição de valores da coluna AUTOR_SEX0:

AUTOR_SEX0

1 203699

2 47310

3 22445

9 20487

452

Name: count, dtype: int64

Distribuição de valores da coluna CONS_TUTEL:

CONS_TUTEL

2 219321

1 61126

9 9449

4497

Name: count, dtype: int64

Distribuição de valores da coluna CS_ESCOL_N:

CS_ESCOL_N

09 63231

```
06    53551
03    40490
10    34027
05    27037
01    17265
04    16970
08    12185
      9892
07     8854
02     8458
00     2433
Name: count, dtype: int64
```

Distribuição de valores da coluna CS_RACA:

```
CS_RACA
4    139105
1    110208
2     29672
9      8680
5      3252
3      2917
      559
Name: count, dtype: int64
```

Distribuição de valores da coluna CS_SEX0:

```
CS_SEX0
F    294393
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_AUDITI:

```
DEF_AUDITI
8    266701
2    23231
      2827
1      858
9      776
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_FISICA:

```
DEF_FISICA
8    266693
2    21300
1     2908
      2770
9      722
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_MENTAL:

```
DEF_MENTAL
8    266718
2    20563
```

```
1      3453
      2805
9      854
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_OUT:

```
DEF_OUT
8      266694
2      19500
1       3922
      3087
9       1190
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_TRANS:

```
DEF_TRANS
2      232528
9      36698
1      23384
      1561
8        222
Name: count, dtype: int64
```

Distribuição de valores da coluna DEF_VISUAL:

```
DEF_VISUAL
8      266704
2      23039
      2815
1       1052
9        783
Name: count, dtype: int64
```

Distribuição de valores da coluna DT_ENCERRA:

```
DT_ENCERRA
20240930      1453
20240909      1386
20240610      1356
20241111      1342
20241125      1329
...
20230326        1
20231231        1
20231222        1
20250411        1
20250501        1
Name: count, Length: 518, dtype: int64
```

Distribuição de valores da coluna DT_OCOR:

```
DT_OCOR
20240101      1927
20240901      1192
```

```
20241006    1123
20240908    1098
20240310    1076
...
20001206      1
20200118      1
19870709      1
19840313      1
19670402      1
Name: count, Length: 4059, dtype: int64
```

```
Distribuição de valores da coluna IDENT_GEN:
IDENT_GEN
8      217815
9       70639
        2745
2       2554
3        634
1         6
Name: count, dtype: int64
```

```
Distribuição de valores da coluna ID_AGRAVO:
ID_AGRAVO
Y09      294393
Name: count, dtype: int64
```

```
Distribuição de valores da coluna LES_AUTOP:
LES_AUTOP
2      294393
Name: count, dtype: int64
```

```
Distribuição de valores da coluna ORIENT_SEX:
ORIENT_SEX
1      175167
8       61017
9       50137
2        4443
3        2396
        1233
Name: count, dtype: int64
```

```
Distribuição de valores da coluna OUT_VEZES:
OUT_VEZES
1      127952
2      112825
9       52644
        971
0         1
Name: count, dtype: int64
```

Distribuição de valores da coluna REDE_SAU:

REDE_SAU

1 176651

2 106605

9 8585

2552

Name: count, dtype: int64

Distribuição de valores da coluna REL_CONHEC:

REL_CONHEC

2 241602

1 37285

9 11597

3909

Name: count, dtype: int64

Distribuição de valores da coluna REL_CONJ:

REL_CONJ

2 222803

1 57054

9 10814

3722

Name: count, dtype: int64

Distribuição de valores da coluna REL_CUIDA:

REL_CUIDA

2 277470

9 11174

4230

1 1519

Name: count, dtype: int64

Distribuição de valores da coluna REL_DESCO:

REL_DESCO

2 249507

1 29421

9 11382

4083

Name: count, dtype: int64

Distribuição de valores da coluna REL_EXCON:

REL_EXCON

2 249500

1 29904

9 10997

3992

Name: count, dtype: int64

Distribuição de valores da coluna REL_EXNAM:

REL_EXNAM

```
2    269588
9     11161
1      9454
      4190
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_FILHO:
REL_FILHO
2    266655
1    12705
9    10843
      4190
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_INST:
REL_INST
2    275915
9     11042
      4229
1      3207
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_IRMAO:
REL_IRMAO
2    269652
9     11332
1      9219
      4190
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_MAD:
REL_MAD
2    278076
9     11191
      4247
1       879
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_MAE:
REL_MAE
2    250845
1     29127
9     11111
      3310
Name: count, dtype: int64
```

```
Distribuição de valores da coluna REL_NAM0:
REL_NAM0
2    263813
1     15292
```

```
9      11165
      4123
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_OUTROS:

```
REL_OUTROS
2      249420
1      28484
9      12373
      4116
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_PAD:

```
REL_PAD
2      269552
9      11330
1       9320
      4191
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_PAI:

```
REL_PAI
2      254500
1      24880
9      11466
      3547
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_PATRAO:

```
REL_PATRAO
2      278184
9      10857
      4241
1       1111
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_POL:

```
REL_POL
2      278311
9      10981
      4234
1       867
Name: count, dtype: int64
```

Distribuição de valores da coluna REL_PROPRI:

```
REL_PROPRI
2      276093
9       9270
1       4832
      4198
```

Name: count, dtype: int64

Distribuição de valores da coluna REL_TRAB:

REL_TRAB

2 276187

9 8779

1 5263

4164

Name: count, dtype: int64

Distribuição de valores da coluna SG_UF:

SG_UF

35 73761

33 36825

41 27538

31 24730

43 16381

23 14031

26 13059

29 10807

42 9780

52 9075

32 8590

15 8306

50 6043

13 5503

27 4458

53 3724

21 3008

51 2956

22 2954

17 2819

28 2226

25 2211

24 1988

14 1161

11 1062

12 866

16 497

34

Name: count, dtype: int64

Distribuição de valores da coluna SIT_CONJUG:

SIT_CONJUG

1 115748

2 67717

8 54686

9 29202

4 17154

3 6417

3469

Name: count, dtype: int64

Distribuição de valores da coluna VIOL_FINAN:
VIOL_FINAN
2 278530
1 9008
3778
9 3077
Name: count, dtype: int64

Distribuição de valores da coluna VIOL_FISIC:
VIOL_FISIC
1 172437
2 118228
1977
9 1751
Name: count, dtype: int64

Distribuição de valores da coluna VIOL_INFAN:
VIOL_INFAN
2 286961
3801
9 2900
1 731
Name: count, dtype: int64

Distribuição de valores da coluna VIOL_LEGAL:
VIOL_LEGAL
2 287056
3795
9 3100
1 442
Name: count, dtype: int64

Distribuição de valores da coluna VIOL_MOTIV:
VIOL_MOTIV
01 82795
99 70787
09 64136
88 40981
06 27917
07 2360
08 1921
1546
02 1184
03 427
04 174
05 165
Name: count, dtype: int64

Distribuição de valores da coluna VIOL_NEGLI:
VIOL_NEGLI

```
2    255468
1     34048
9      2898
      1979
Name: count, dtype: int64
```

```
Distribuição de valores da coluna VIOL_OUTR:
VIOL_OUTR
2    279177
1      6493
9      4896
      3827
Name: count, dtype: int64
```

```
Distribuição de valores da coluna VIOL_PSICO:
VIOL_PSICO
2    199553
1     88503
      3553
9      2784
Name: count, dtype: int64
```

```
Distribuição de valores da coluna VIOL_SEXU:
VIOL_SEXU
2    217812
1     70572
      3360
9      2649
Name: count, dtype: int64
```

```
Distribuição de valores da coluna VIOL_TORT:
VIOL_TORT
2    280623
1      6568
      3725
9      3477
Name: count, dtype: int64
```

```
Distribuição de valores da coluna VIOL_TRAF:
VIOL_TRAF
2    287549
      3798
9      2870
1       176
Name: count, dtype: int64
```

```
In [9]: cleaned_df.loc['DT_OCOR_CONV'] = pd.to_datetime(cleaned_df['DT_OCOR'].astype('datetime64[ns]'))
```

```
cleaned_df.loc['DIA_SEMANA_OCOR'] = cleaned_df['DT_OCOR_CONV'].dt.dayofweek.  
cleaned_df.loc['MES_OCOR'] = cleaned_df['DT_OCOR_CONV'].dt.month.map({1: 'JAN'  
  
cleaned_df[['DT_OCOR', 'DT_OCOR_CONV', 'DIA_SEMANA_OCOR', 'MES_OCOR']].head()  
  
# mostrar a distribuição de valores da coluna MES_OCOR  
print(cleaned_df['MES_OCOR'].value_counts())  
  
# mostrar a distribuição de valores da coluna DIA_SEMANA_OCOR  
print(cleaned_df['DIA_SEMANA_OCOR'].value_counts())  
  
#
```

```
/var/folders/1t/2p42d58j00g_3gn8s1rm7nch0000gn/T/ipykernel_82183/2863901152.  
py:1: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
cleaned_df.loc['DT_OCOR_CONV'] = pd.to_datetime(cleaned_df['DT_OCOR']).astype(  
str, format='%Y%m%d', errors='coerce')
```

```

-----
KeyError                                Traceback (most recent call last)
File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexes/base.p
y:3812, in Index.get_loc(self, key)
    3811 try:
-> 3812     return self._engine.get_loc(casted_key)
    3813 except KeyError as err:

File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()

File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtabl
e.PyObjectHashTable.get_item()

File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtabl
e.PyObjectHashTable.get_item()

```

KeyError: 'DT_OCOR_CONV'

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
Cell In[9], line 4
      1 cleaned_df.loc['DT_OCOR_CONV'] = pd.to_datetime(cleaned_df['DT_OCOR
R']).astype(str), format='%Y%m%d', errors='coerce')
----> 4 cleaned_df.loc['DIA_SEMANA_OCOR'] = cleaned_df['DT_OCOR_CONV'].dt.da
yofweek.map({0: 'SEG', 1: 'TER', 2: 'QUA', 3: 'QUI', 4: 'SEX', 5: 'SAB', 6: 'DOM'})
      5 cleaned_df.loc['MES_OCOR'] = cleaned_df['DT_OCOR_CONV'].dt.month.map
({1: 'JAN', 2: 'FEV', 3: 'MAR', 4: 'ABR', 5: 'MAI', 6: 'JUN', 7: 'JUL', 8: 'AGO', 9: 'SET', 1
0: 'OUT', 11: 'NOV', 12: 'DEZ'})
      7 cleaned_df[['DT_OCOR', 'DT_OCOR_CONV', 'DIA_SEMANA_OCOR', 'MES_OCOR']].
head()

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/frame.py:4113,
in DataFrame.__getitem__(self, key)
    4111 if self.columns.nlevels > 1:
    4112     return self._getitem_multilevel(key)
-> 4113 indexer = self.columns.get_loc(key)
    4114 if is_integer(indexer):
    4115     indexer = [indexer]

File /opt/anaconda3/lib/python3.13/site-packages/pandas/core/indexes/base.p
y:3819, in Index.get_loc(self, key)
    3814 if isinstance(casted_key, slice) or (
    3815     isinstance(casted_key, abc.Iterable)
    3816     and any(isinstance(x, slice) for x in casted_key)
    3817 ):
    3818     raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
    3820 except TypeError:
    3821     # If we have a listlike key, _check_indexing_error will raise
    3822     # InvalidIndexError. Otherwise we fall through and re-raise
    3823     # the TypeError.
    3824     self._check_indexing_error(key)

```

```
KeyError: 'DT_OCOR_CONV'
```

Histogramas e Box-plot

```
In [ ]: pip install numpy matplotlib
```

```
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.13/site-packages (2.2.6)
Requirement already satisfied: matplotlib in /opt/anaconda3/lib/python3.13/site-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (1.3.1)
Requirement already satisfied: cyclor>=0.10 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/python3.13/site-packages (from matplotlib) (2.8.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

TOP_N = 20

num_cols = cleaned_df.select_dtypes(include=np.number).columns
cat_cols = cleaned_df.columns.difference(num_cols)

def plot_col(c):
    s = cleaned_df[c].dropna()
    if s.empty:
        return

    sn = pd.to_numeric(s, errors="coerce").dropna()

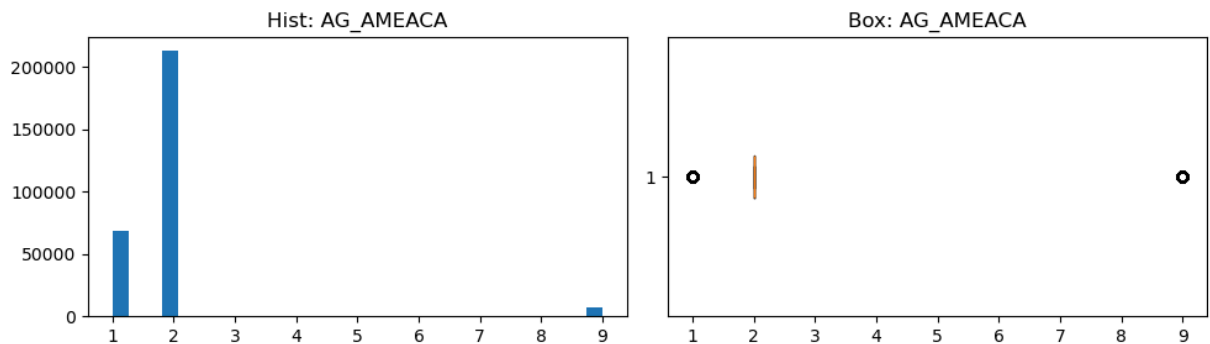
    if not sn.empty: # numérica
        fig, ax = plt.subplots(1, 2, figsize=(10, 3))
        ax[0].hist(sn, bins=30)
        ax[0].set_title(f"Hist: {c}")
        ax[1].boxplot(sn, vert=False)
        ax[1].set_title(f"Box: {c}")
        plt.tight_layout()
        plt.show()
```

```

    print(c, sn.describe())
else: # categórica
    vc = s.astype(str).str.strip().replace({"": np.nan}).value_counts(dropna=False)
    vc = vc.sort_values(ascending=True) # barh: maior em cima
    fig, ax = plt.subplots(figsize=(10, 0.35*len(vc)+1))
    bars = ax.barh(vc.index.astype(str), vc.values)
    ax.set_title(f"{c} (top {TOP_N})")
    ax.bar_label(bars, padding=3, fontsize=9)
    plt.tight_layout()
    plt.show()
    print(c, s.describe())

# roda pra todas (ou use cleaned_df.columns[:10] pra testar)
for c in cleaned_df.columns:
    plot_col(c)

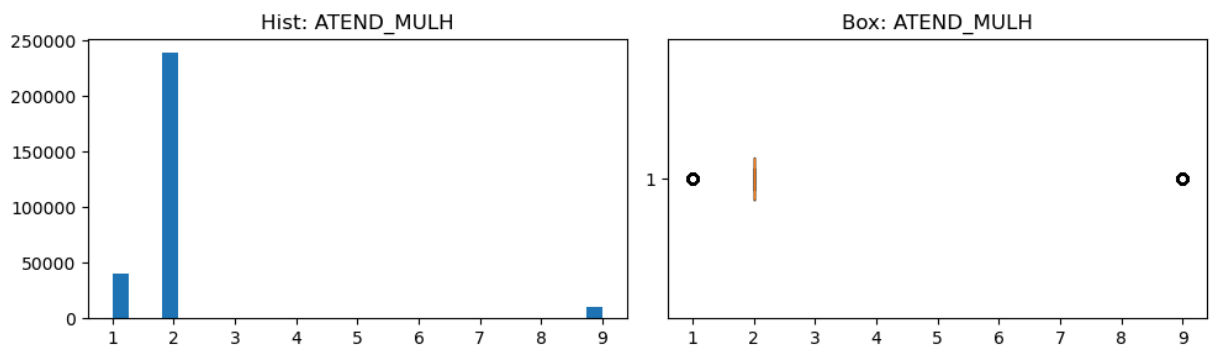
```



```

AG_AMEACA count      288630.000000
mean              1.931559
std               1.189013
min              1.000000
25%              2.000000
50%              2.000000
75%              2.000000
max              9.000000
Name: AG_AMEACA, dtype: float64

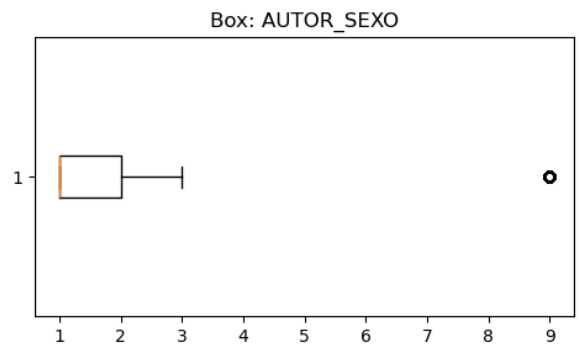
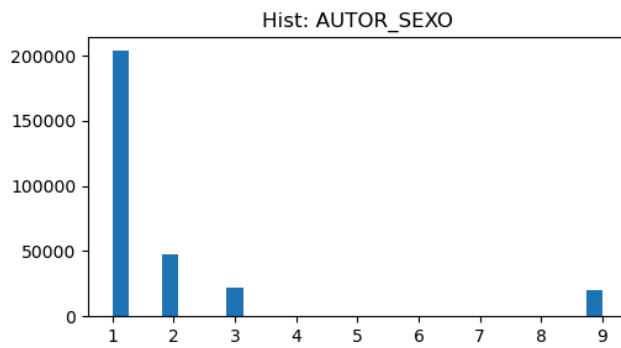
```



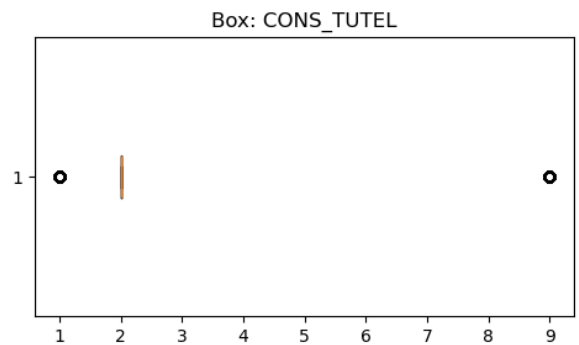
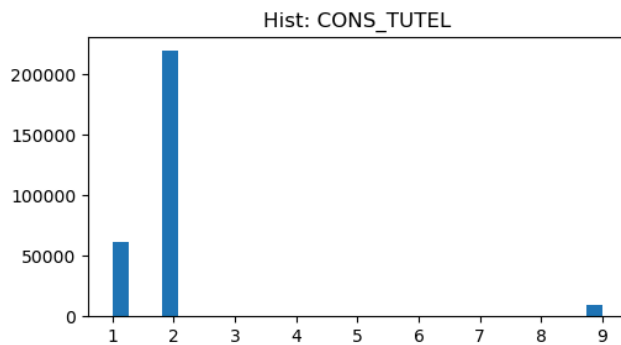
```

ATEND_MULH count      289921.000000
mean              2.112272
std               1.372149
min              1.000000
25%              2.000000
50%              2.000000
75%              2.000000
max              9.000000
Name: ATEND_MULH, dtype: float64

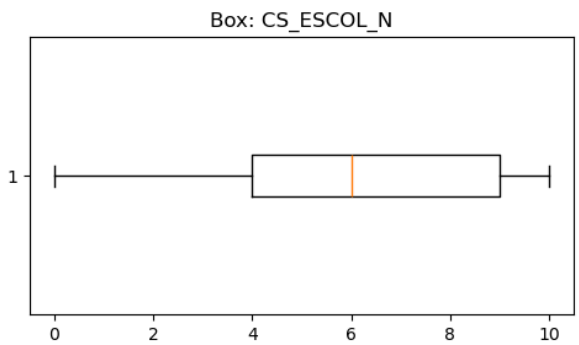
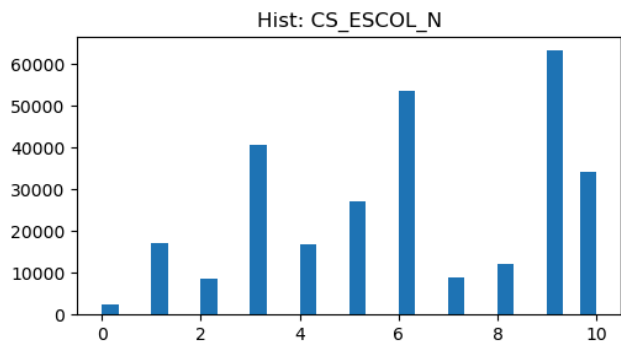
```



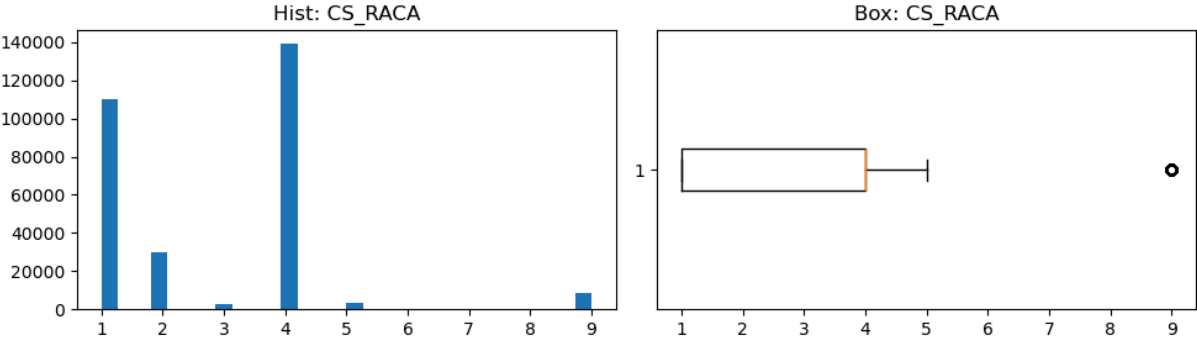
```
AUTOR_SEXO count    293941.000000
mean                1.871250
std                 2.041562
min                 1.000000
25%                 1.000000
50%                 1.000000
75%                 2.000000
max                 9.000000
Name: AUTOR_SEXO, dtype: float64
```



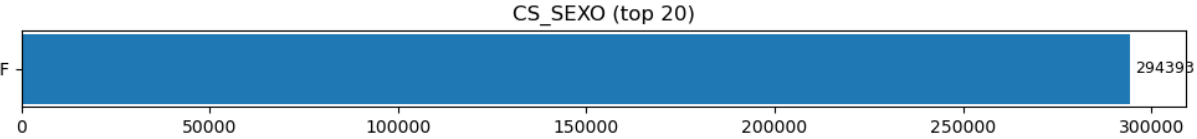
```
CONS_TUTEL count    289896.000000
mean                2.017306
std                 1.344503
min                 1.000000
25%                 2.000000
50%                 2.000000
75%                 2.000000
max                 9.000000
Name: CONS_TUTEL, dtype: float64
```



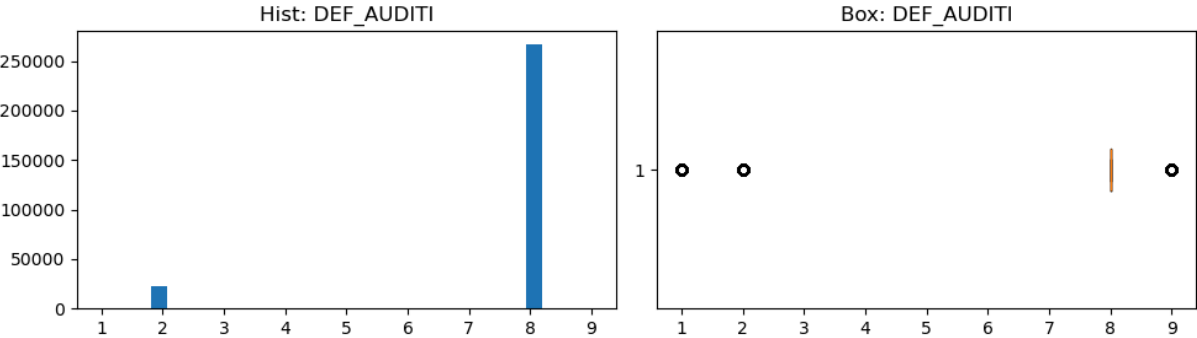
CS_ESCOL_N count 284501.000000
mean 6.147005
std 2.830182
min 0.000000
25% 4.000000
50% 6.000000
75% 9.000000
max 10.000000
Name: CS_ESCOL_N, dtype: float64



CS_RACA count 293834.000000
mean 2.821671
std 1.774995
min 1.000000
25% 1.000000
50% 4.000000
75% 4.000000
max 9.000000
Name: CS_RACA, dtype: float64

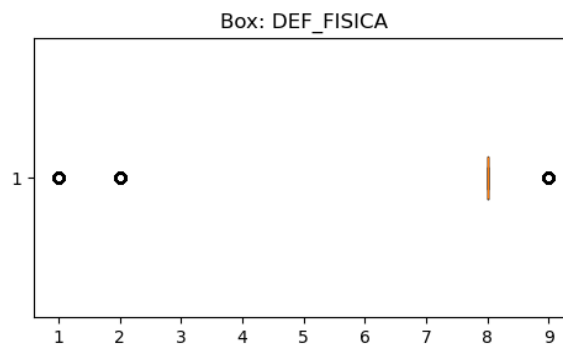
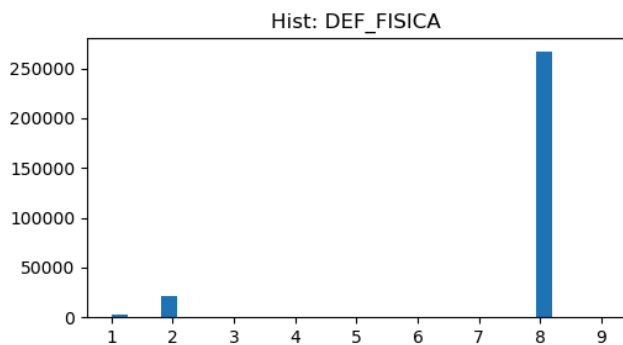


CS_SEX0 count 294393
unique 1
top F
freq 294393
Name: CS_SEX0, dtype: object



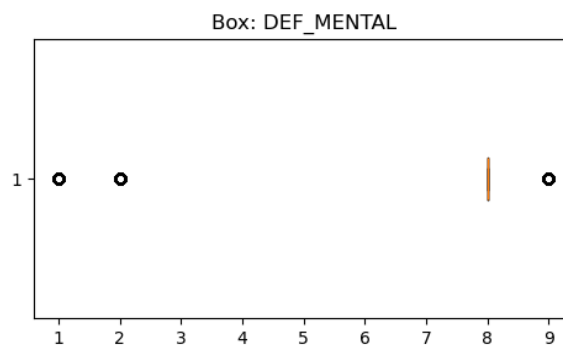
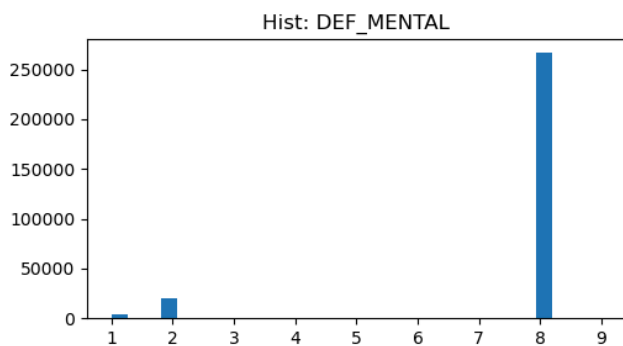
DEF_AUDITI count 291566.000000
mean 7.504003
std 1.664094
min 1.000000
25% 8.000000
50% 8.000000
75% 8.000000
max 9.000000

Name: DEF_AUDITI, dtype: float64



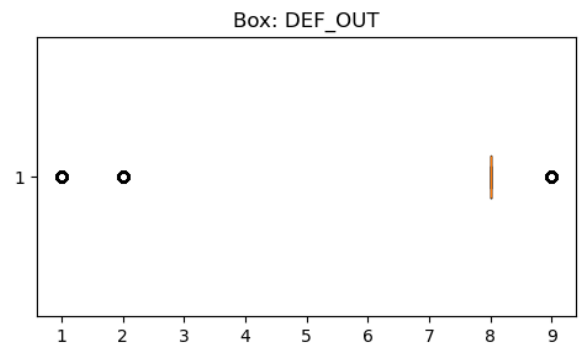
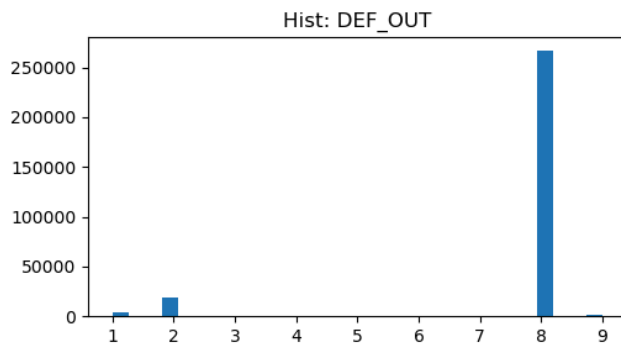
DEF_FISICA count 291623.000000
mean 7.494436
std 1.692611
min 1.000000
25% 8.000000
50% 8.000000
75% 8.000000
max 9.000000

Name: DEF_FISICA, dtype: float64

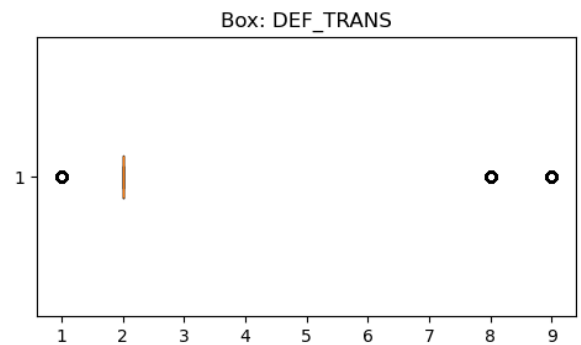
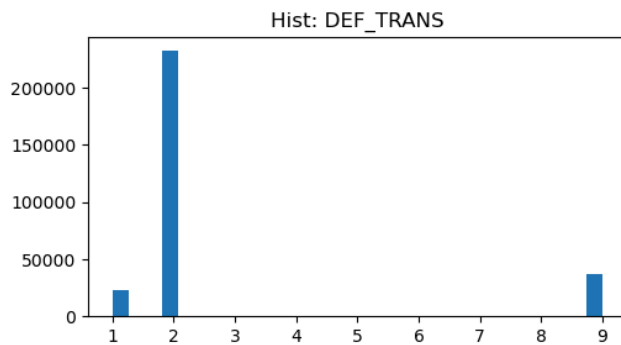


DEF_MENTAL count 291588.000000
mean 7.496910
std 1.693767
min 1.000000
25% 8.000000
50% 8.000000
75% 8.000000
max 9.000000

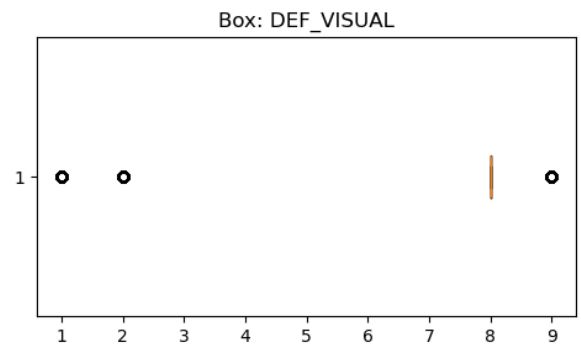
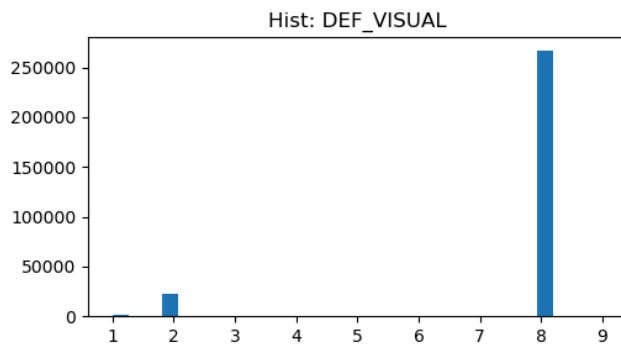
Name: DEF_MENTAL, dtype: float64



```
DEF_OUT count    291306.000000
mean           7.508201
std            1.682789
min            1.000000
25%            8.000000
50%            8.000000
75%            8.000000
max            9.000000
Name: DEF_OUT, dtype: float64
```



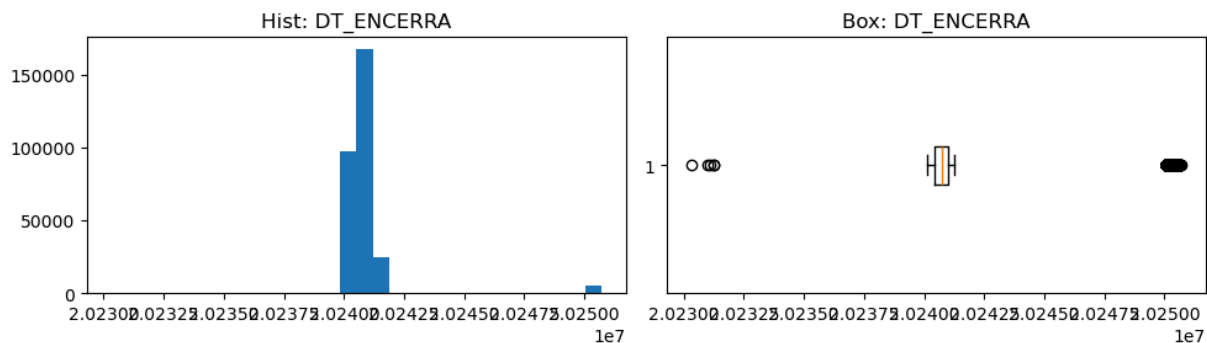
```
DEF_TRANS count    292832.000000
mean           2.801941
std            2.367443
min            1.000000
25%            2.000000
50%            2.000000
75%            2.000000
max            9.000000
Name: DEF_TRANS, dtype: float64
```



```

DEF_VISUAL count    291578.000000
mean                7.503340
std                 1.666538
min                 1.000000
25%                 8.000000
50%                 8.000000
75%                 8.000000
max                 9.000000
Name: DEF_VISUAL, dtype: float64

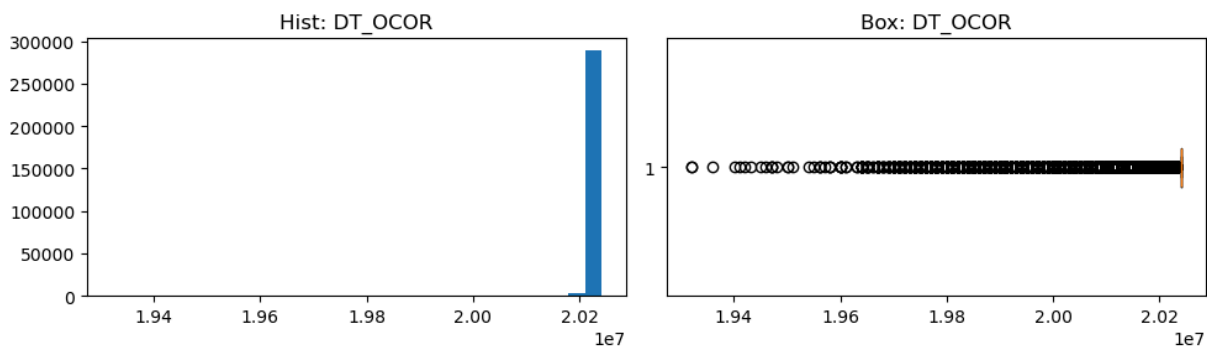
```



```

DT_ENCERRA count    2.934430e+05
mean                2.024085e+07
std                 1.266809e+03
min                 2.023033e+07
25%                 2.024041e+07
50%                 2.024072e+07
75%                 2.024101e+07
max                 2.025070e+07
Name: DT_ENCERRA, dtype: float64

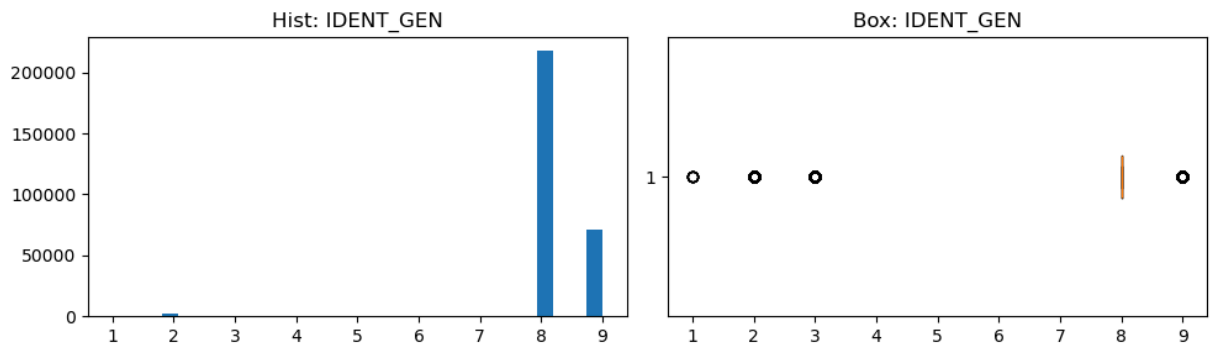
```



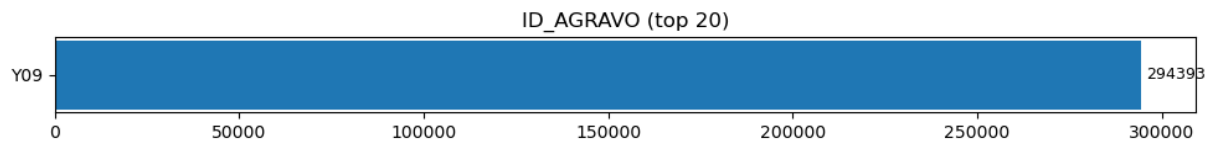
```

DT_OCOR count       2.943930e+05
mean                2.023781e+07
std                 2.354378e+04
min                 1.932071e+07
25%                 2.024031e+07
50%                 2.024062e+07
75%                 2.024092e+07
max                 2.024123e+07
Name: DT_OCOR, dtype: float64

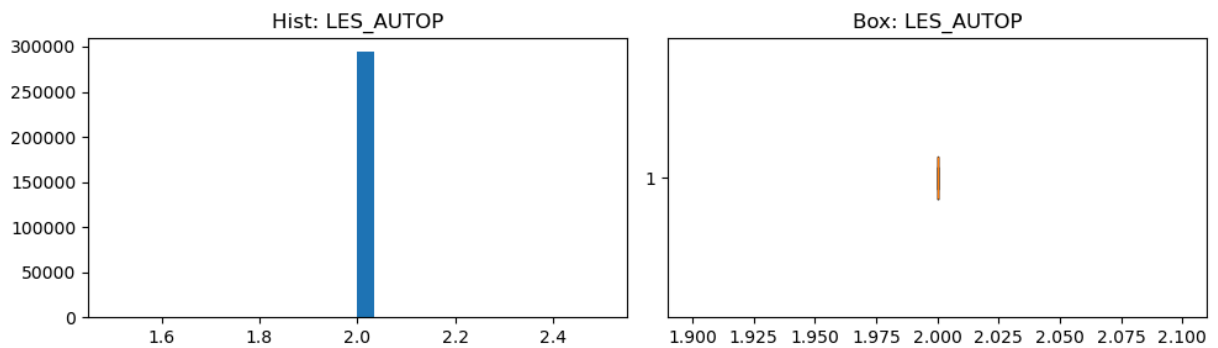
```



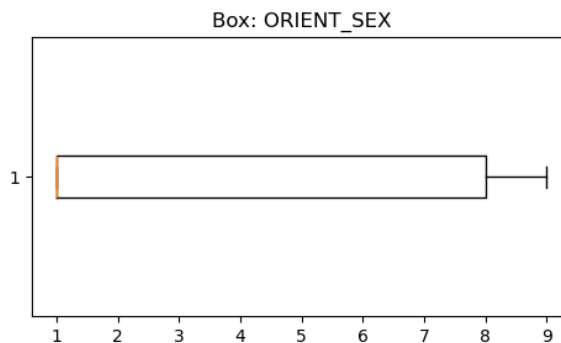
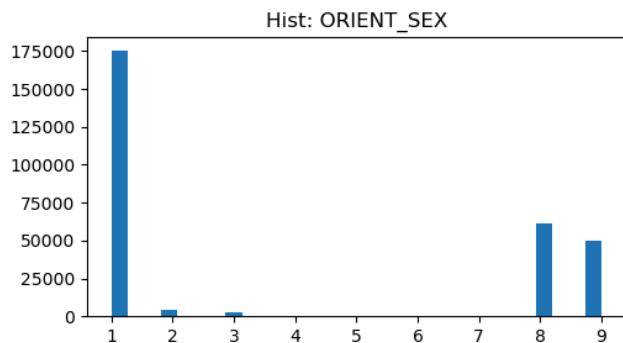
```
IDENT_GEN count    291648.00000
mean           8.17865
std            0.76217
min            1.00000
25%            8.00000
50%            8.00000
75%            8.00000
max            9.00000
Name: IDENT_GEN, dtype: float64
```



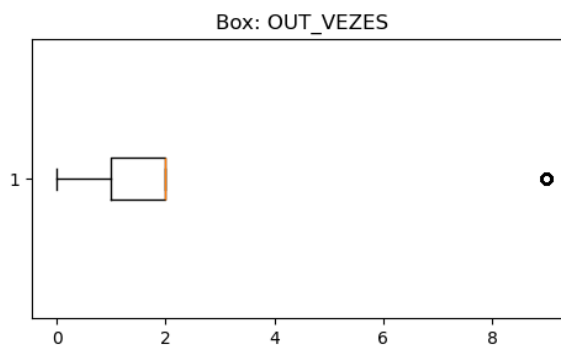
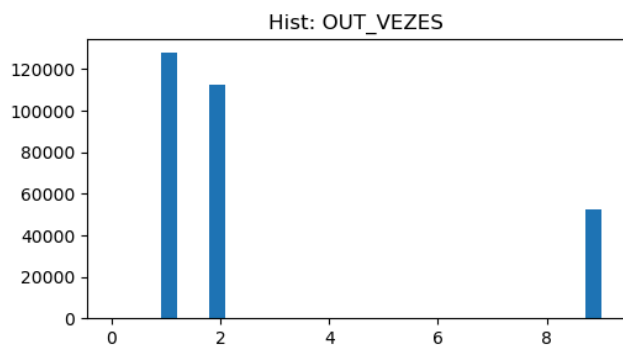
```
ID_AGRAVO count    294393
unique            1
top              Y09
freq            294393
Name: ID_AGRAVO, dtype: object
```



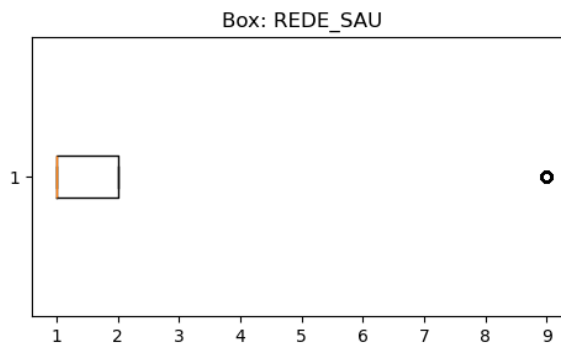
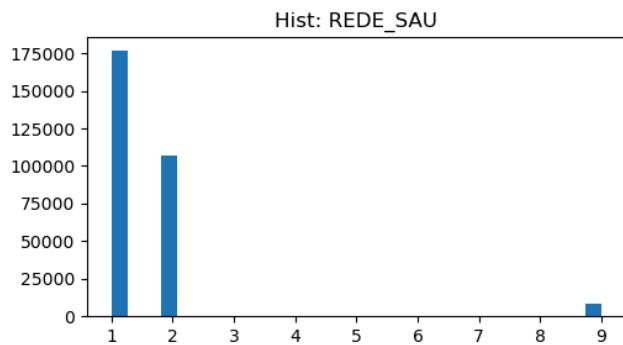
```
LES_AUTOP count    294393.0
mean            2.0
std             0.0
min             2.0
25%             2.0
50%             2.0
75%             2.0
max             2.0
Name: LES_AUTOP, dtype: float64
```



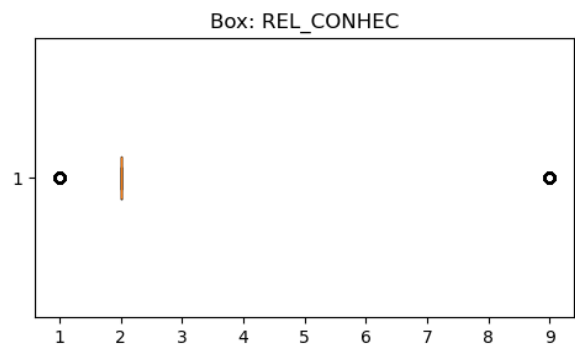
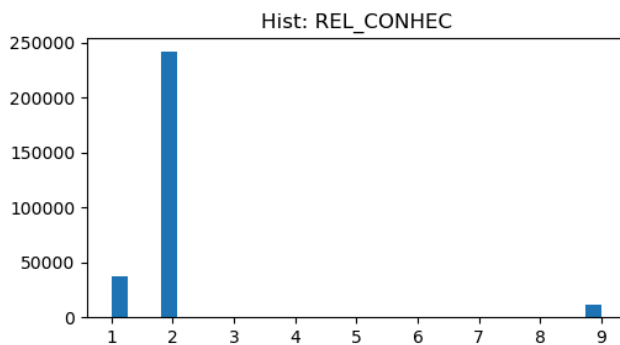
ORIENT_SEX count 293160.000000
mean 3.856631
std 3.609936
min 1.000000
25% 1.000000
50% 1.000000
75% 8.000000
max 9.000000
Name: ORIENT_SEX, dtype: float64



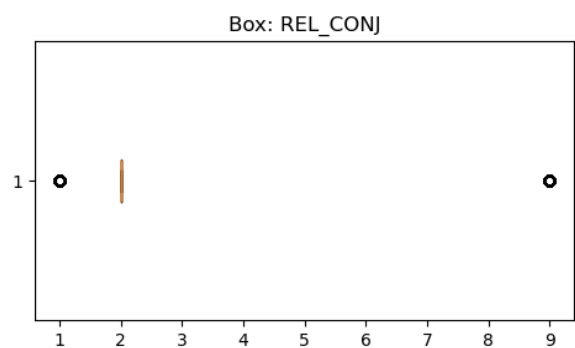
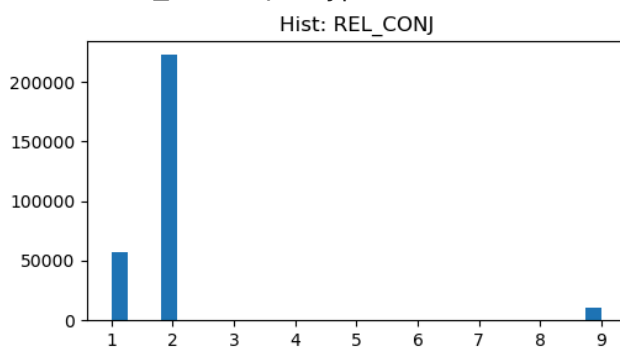
OUT_VEZES count 293422.000000
mean 2.819823
std 2.924942
min 0.000000
25% 1.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: OUT_VEZES, dtype: float64



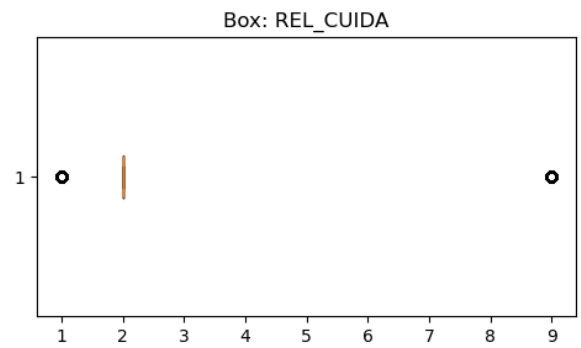
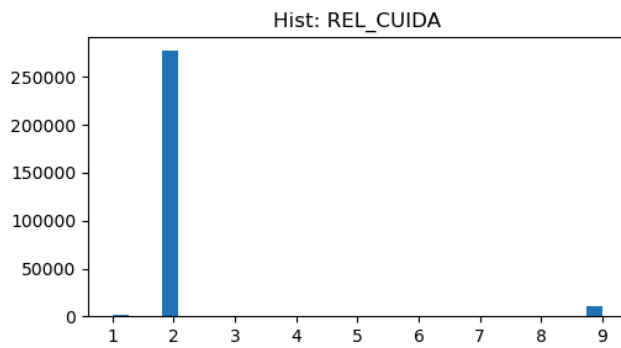
REDE_SAU count 291841.000000
 mean 1.600618
 std 1.373760
 min 1.000000
 25% 1.000000
 50% 1.000000
 75% 2.000000
 max 9.000000
 Name: REDE_SAU, dtype: float64



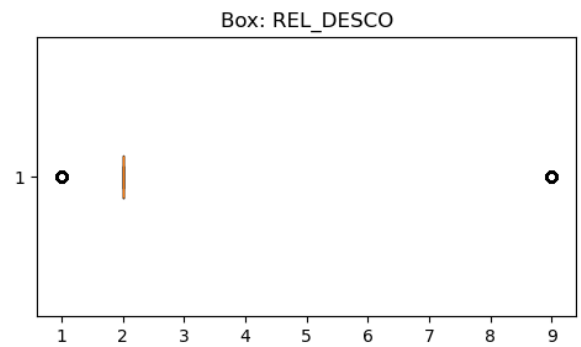
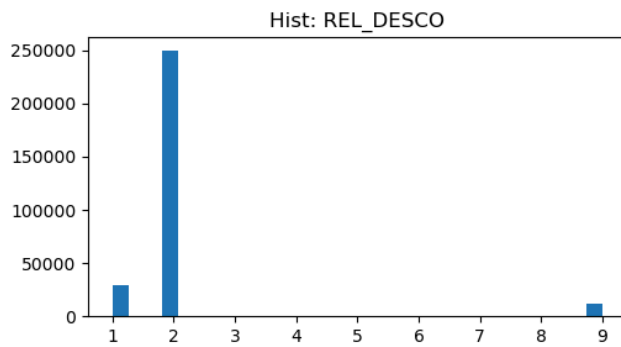
REL_CONHEC count 290484.000000
 mean 2.151106
 std 1.435882
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: REL_CONHEC, dtype: float64



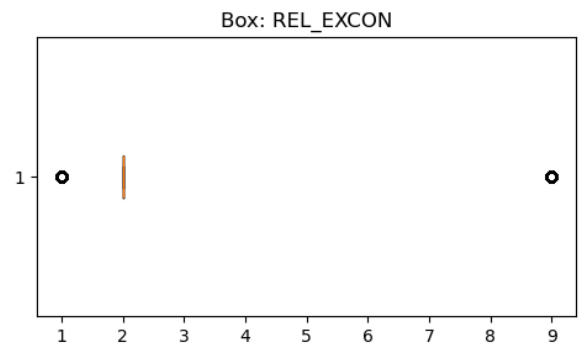
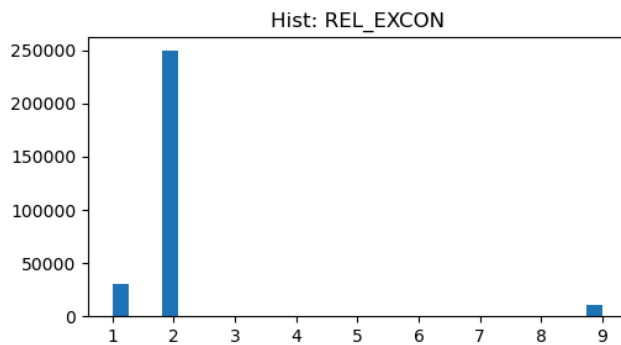
REL_CONJ count 290671.000000
 mean 2.064141
 std 1.419560
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: REL_CONJ, dtype: float64



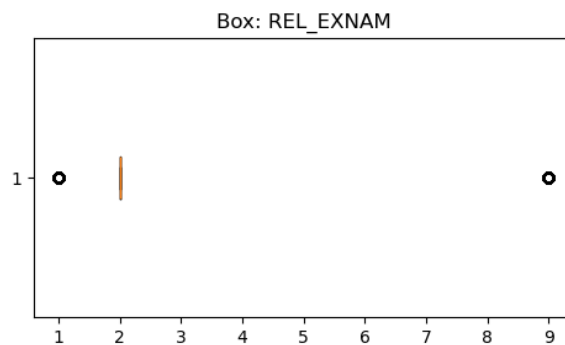
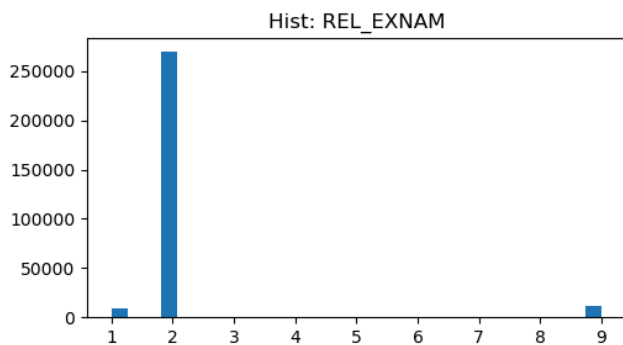
REL_CUIDA count 290163.000000
mean 2.264331
std 1.349937
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_CUIDA, dtype: float64



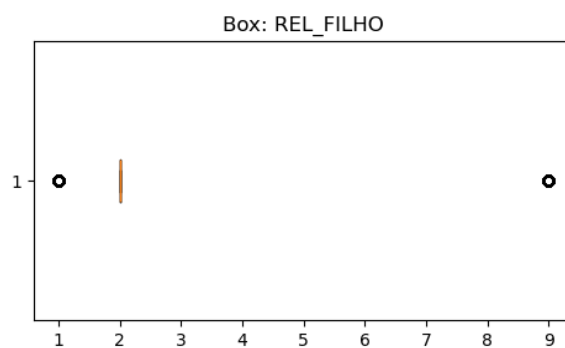
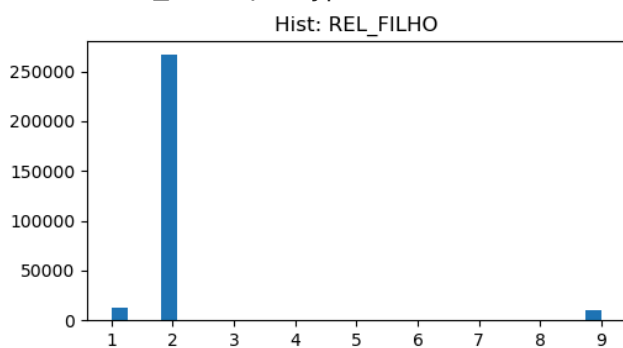
REL_DESCO count 290310.000000
mean 2.173101
std 1.411559
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_DESCO, dtype: float64



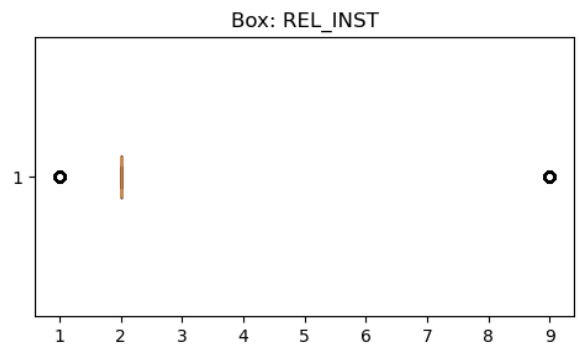
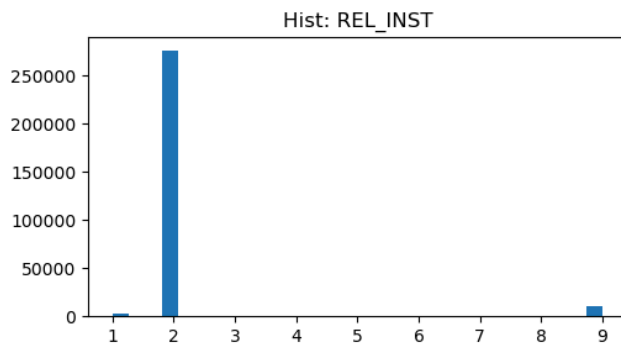
REL_EXCON count 290401.000000
 mean 2.162103
 std 1.390055
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: REL_EXCON, dtype: float64



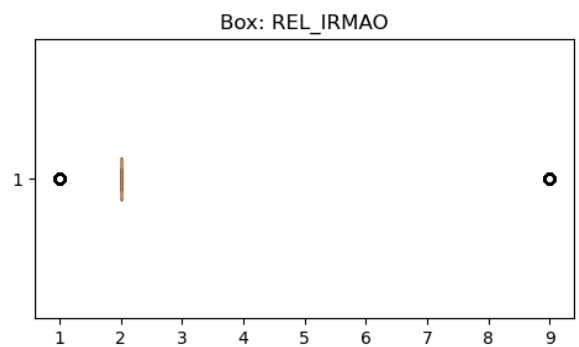
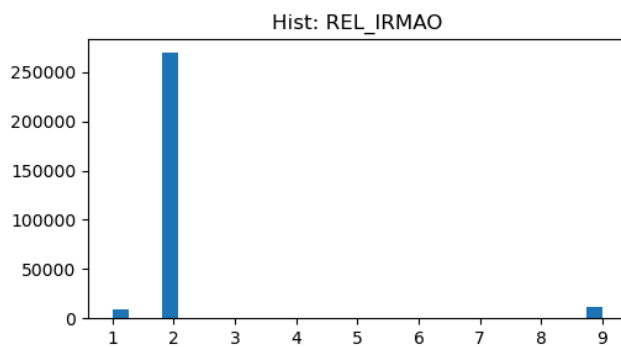
REL_EXNAM count 290203.000000
 mean 2.236638
 std 1.364218
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: REL_EXNAM, dtype: float64



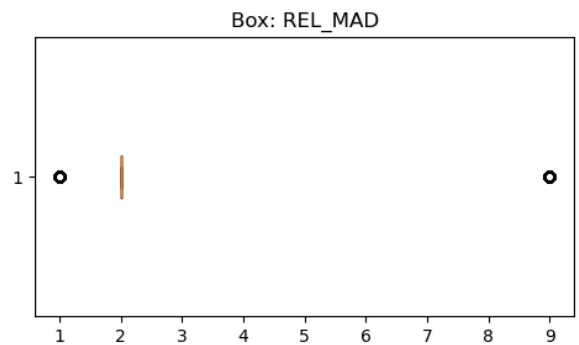
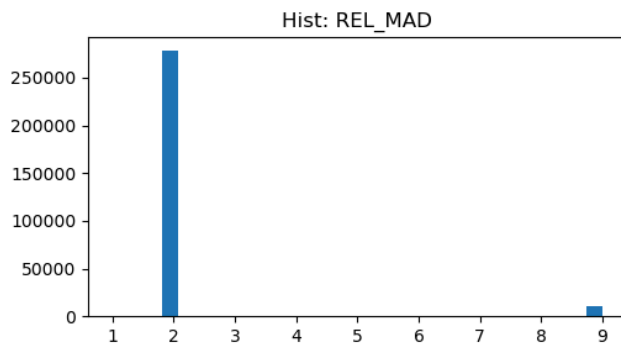
REL_FILHO count 290203.000000
 mean 2.217765
 std 1.351731
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: REL_FILHO, dtype: float64



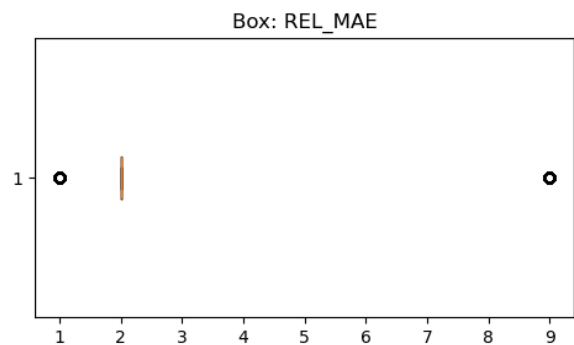
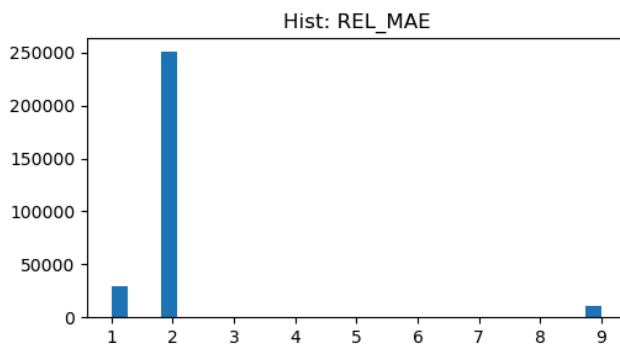
REL_INST count 290164.000000
mean 2.255328
std 1.345559
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_INST, dtype: float64



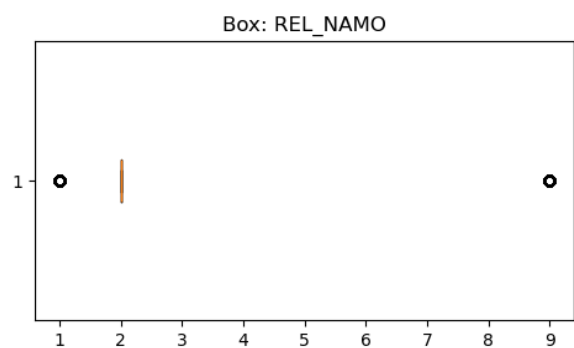
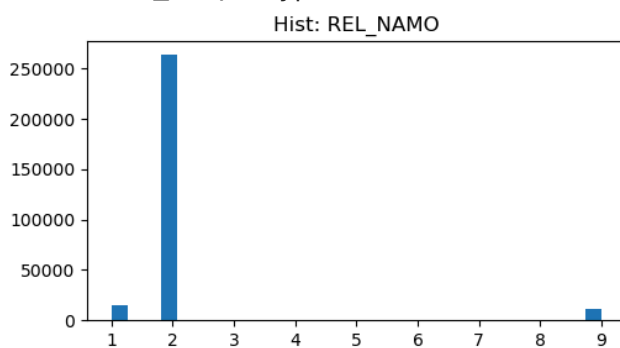
REL_IRMAO count 290203.000000
mean 2.241572
std 1.373606
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_IRMAO, dtype: float64



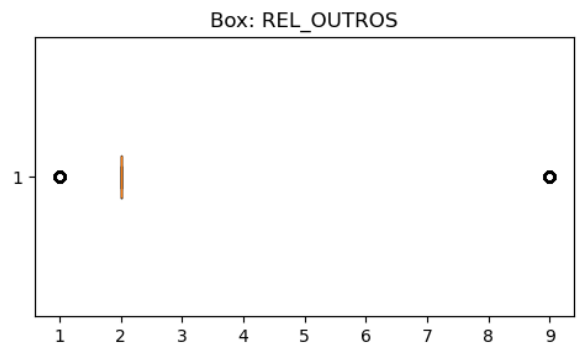
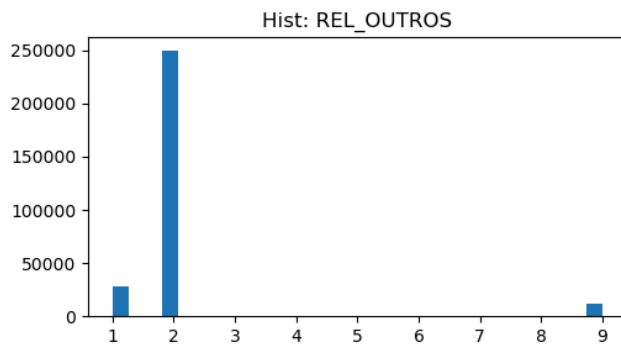
REL_MAD count 290146.000000
mean 2.266962
std 1.349707
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_MAD, dtype: float64



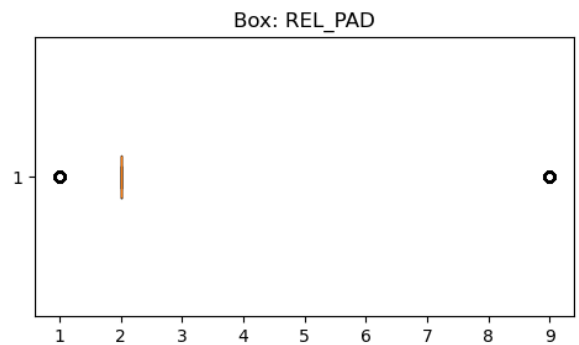
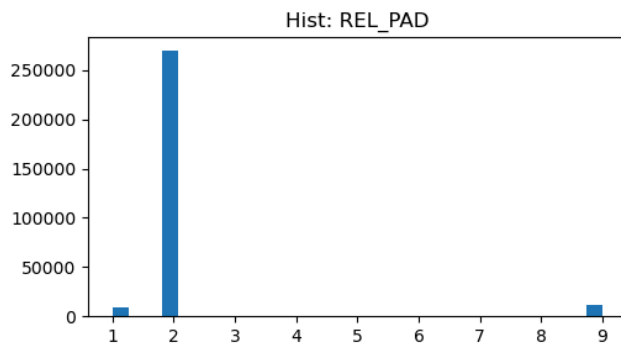
REL_MAE count 291083.000000
mean 2.167134
std 1.393746
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_MAE, dtype: float64



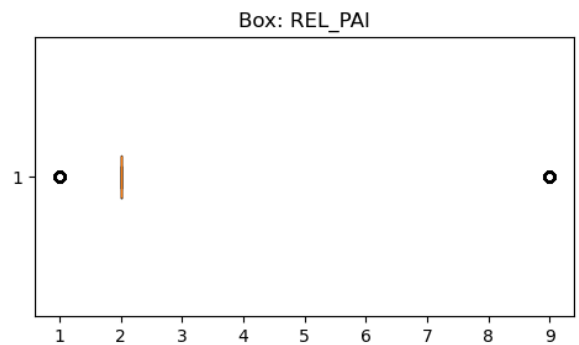
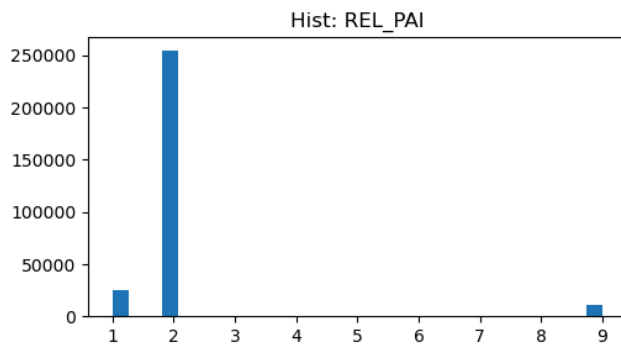
REL_NAMO count 290270.000000
mean 2.216567
std 1.374966
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_NAMO, dtype: float64



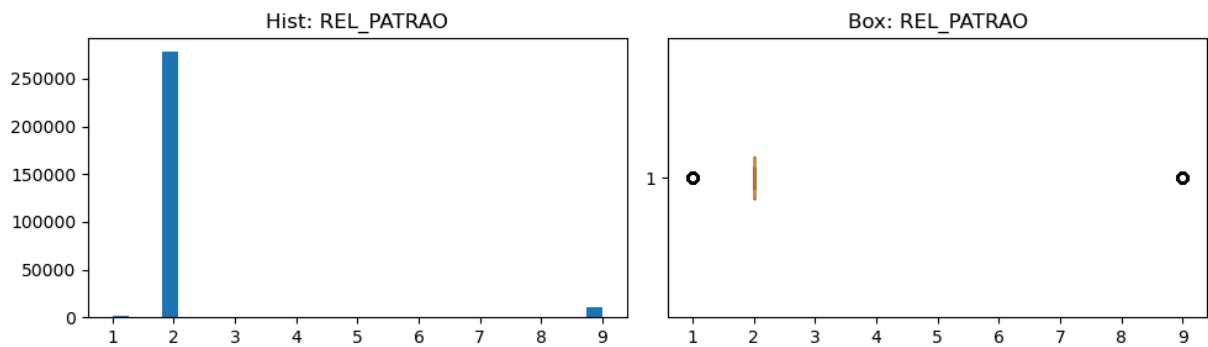
```
REL_OUTROS count    290277.000000
mean                2.200247
std                 1.465145
min                 1.000000
25%                 2.000000
50%                 2.000000
75%                 2.000000
max                 9.000000
Name: REL_OUTROS, dtype: float64
```



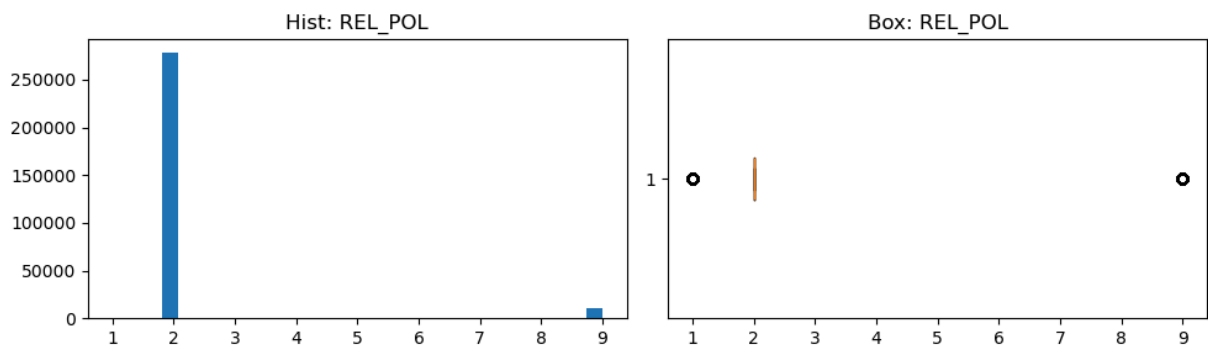
```
REL_PAD count      290202.000000
mean                2.241177
std                 1.373682
min                 1.000000
25%                 2.000000
50%                 2.000000
75%                 2.000000
max                 9.000000
Name: REL_PAD, dtype: float64
```



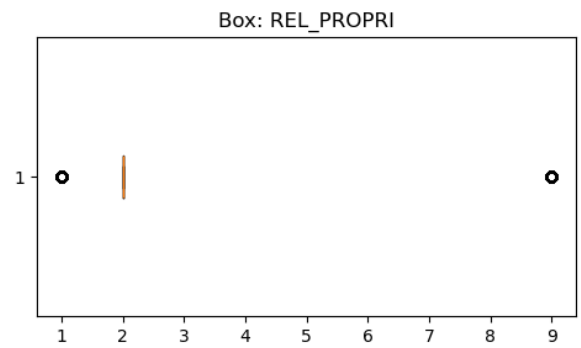
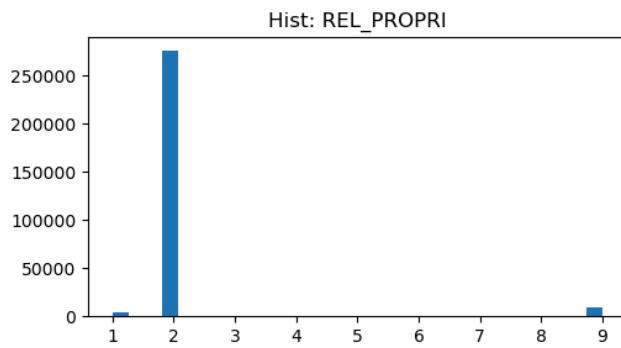
REL_PA1 count 290846.000000
mean 2.190417
std 1.407485
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_PA1, dtype: float64



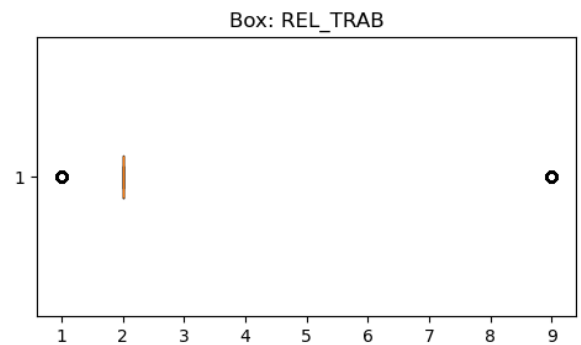
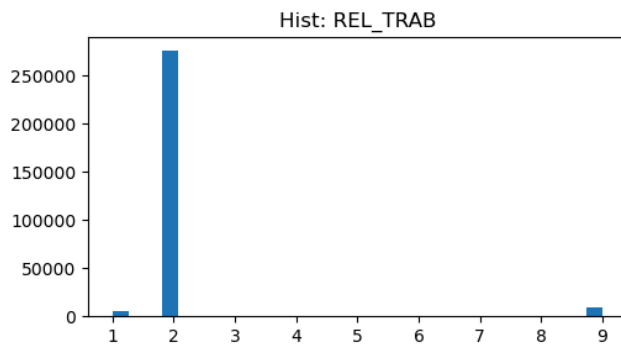
REL_PATRA0 count 290152.000000
mean 2.258099
std 1.330683
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_PATRA0, dtype: float64



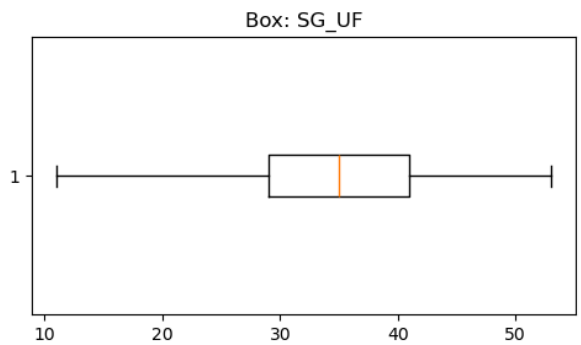
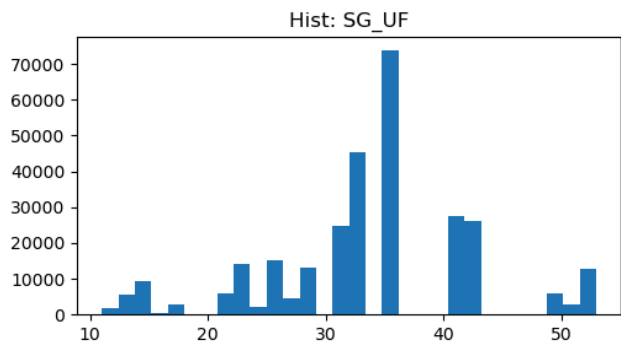
REL_POL count 290159.000000
mean 2.261925
std 1.337454
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: REL_POL, dtype: float64



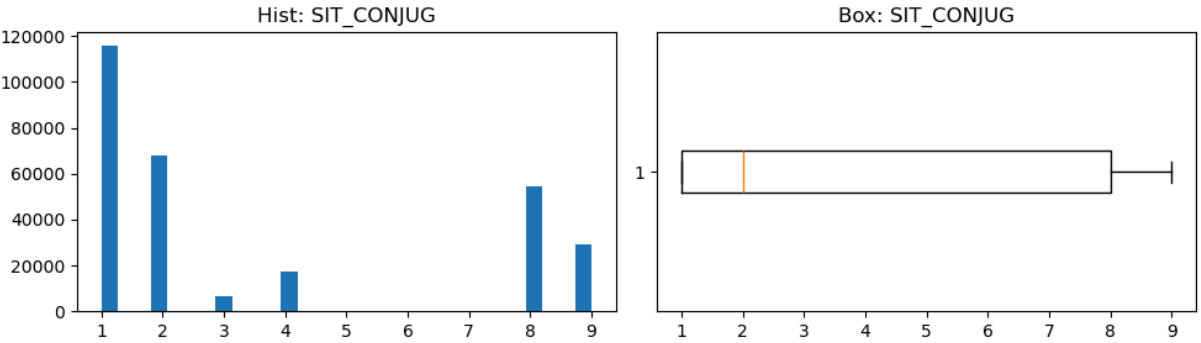
```
REL_PROPRI count    290195.000000
mean              2.206957
std               1.240598
min              1.000000
25%              2.000000
50%              2.000000
75%              2.000000
max              9.000000
Name: REL_PROPRI, dtype: float64
```



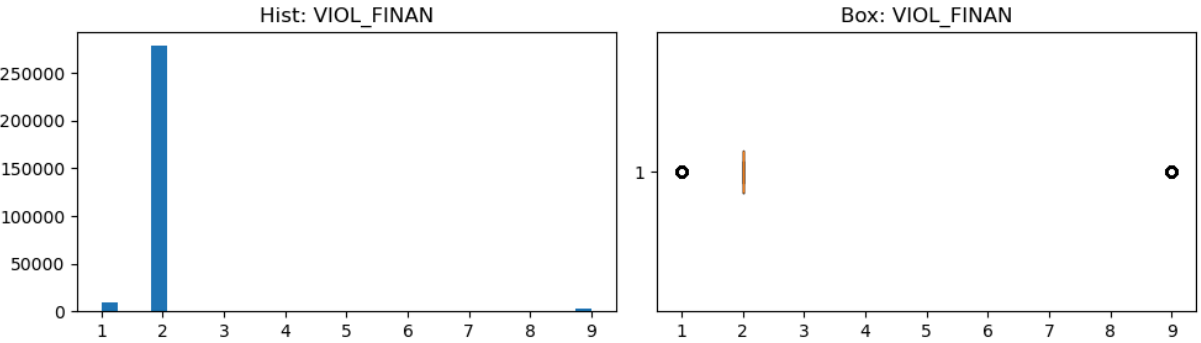
```
REL_TRAB count    290229.000000
mean              2.193606
std               1.209477
min              1.000000
25%              2.000000
50%              2.000000
75%              2.000000
max              9.000000
Name: REL_TRAB, dtype: float64
```



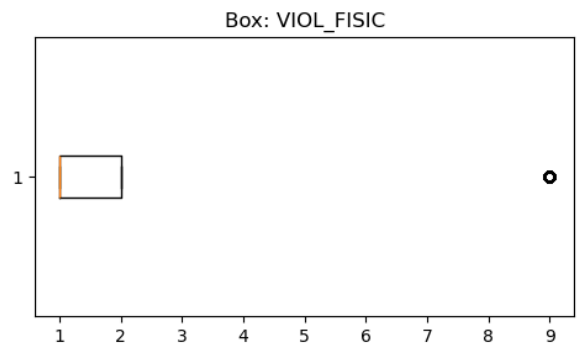
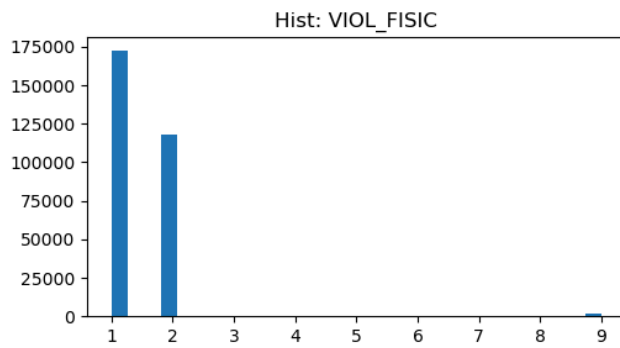
SG_UF count 294359.000000
mean 33.579975
std 8.755269
min 11.000000
25% 29.000000
50% 35.000000
75% 41.000000
max 53.000000
Name: SG_UF, dtype: float64



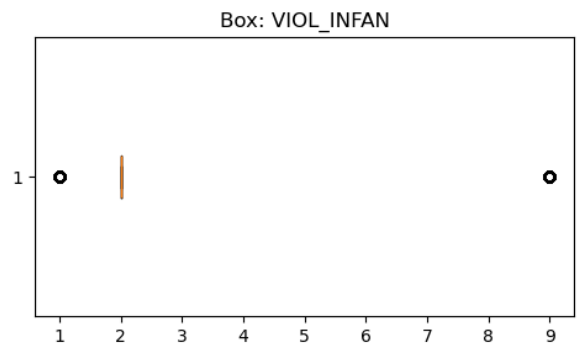
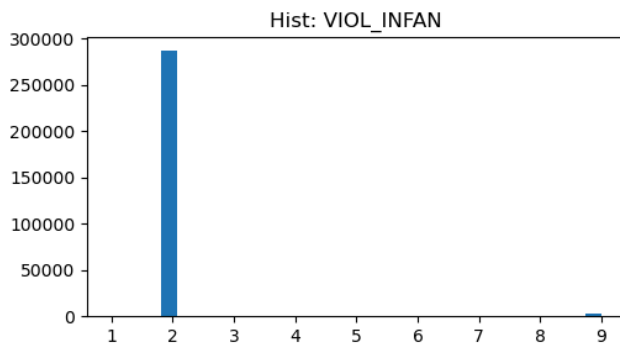
SIT_CONJUG count 290924.000000
mean 3.572600
std 3.141376
min 1.000000
25% 1.000000
50% 2.000000
75% 8.000000
max 9.000000
Name: SIT_CONJUG, dtype: float64



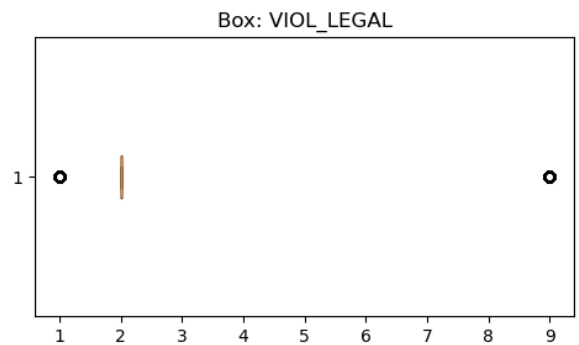
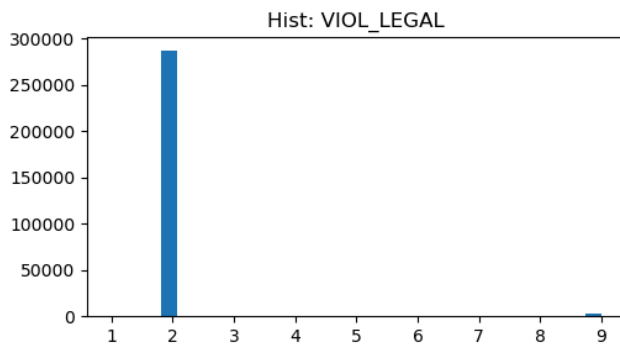
VIOL_FINAN count 290615.000000
mean 2.043119
std 0.740234
min 1.000000
25% 2.000000
50% 2.000000
75% 2.000000
max 9.000000
Name: VIOL_FINAN, dtype: float64



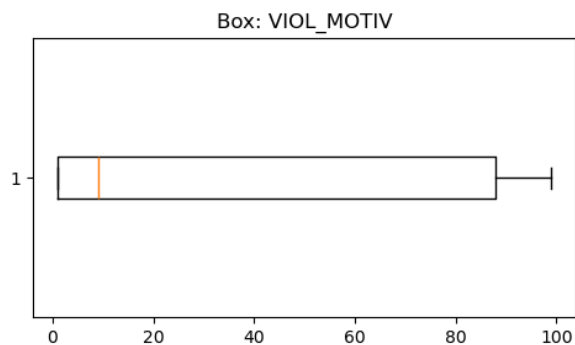
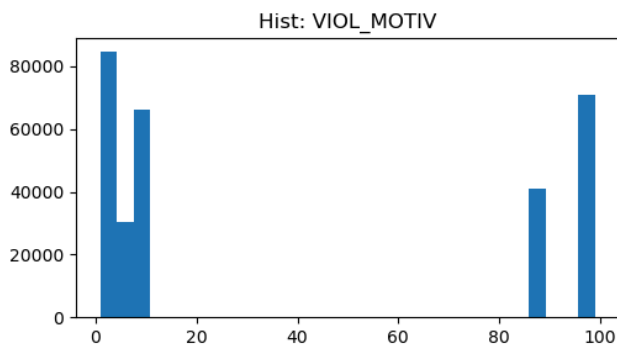
```
VIOL_FISIC count    292416.000000
mean              1.452219
std               0.763577
min              1.000000
25%              1.000000
50%              1.000000
75%              2.000000
max              9.000000
Name: VIOL_FISIC, dtype: float64
```



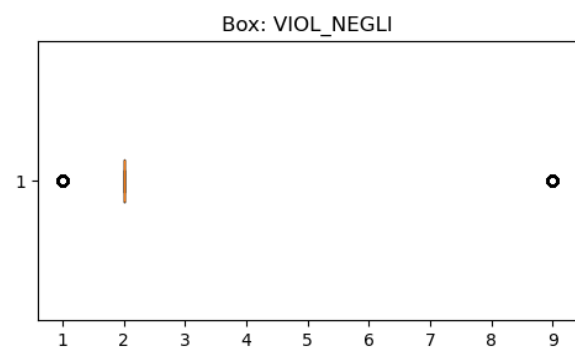
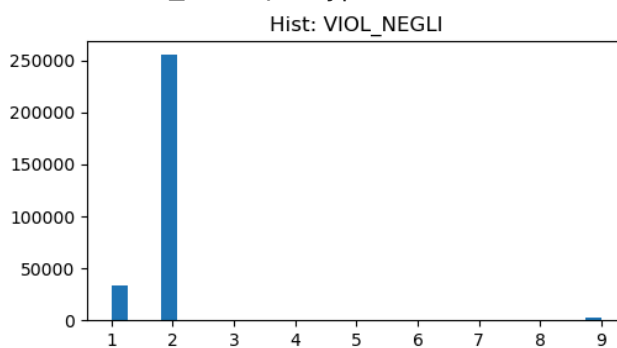
```
VIOL_INFAN count    290592.000000
mean              2.067342
std               0.697842
min              1.000000
25%              2.000000
50%              2.000000
75%              2.000000
max              9.000000
Name: VIOL_INFAN, dtype: float64
```



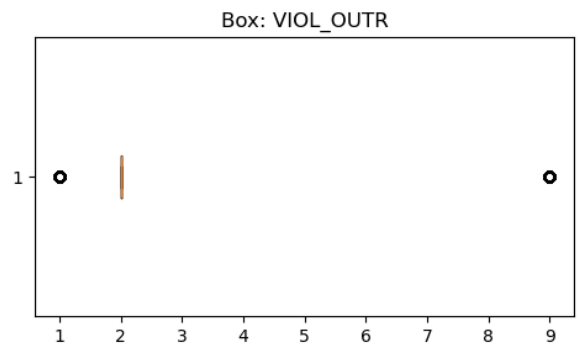
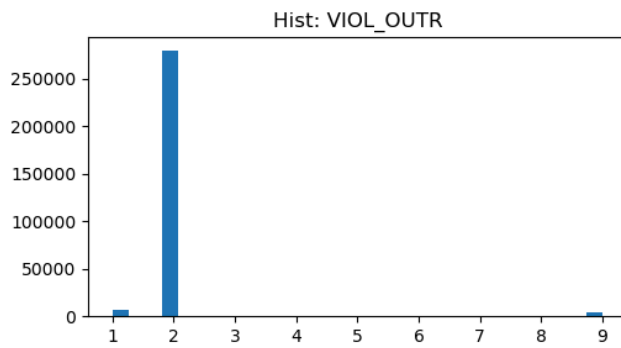
VIOL_LEGAL count 290598.000000
 mean 2.073153
 std 0.720338
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_LEGAL, dtype: float64



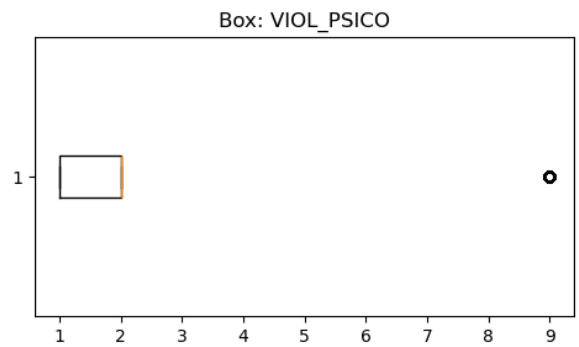
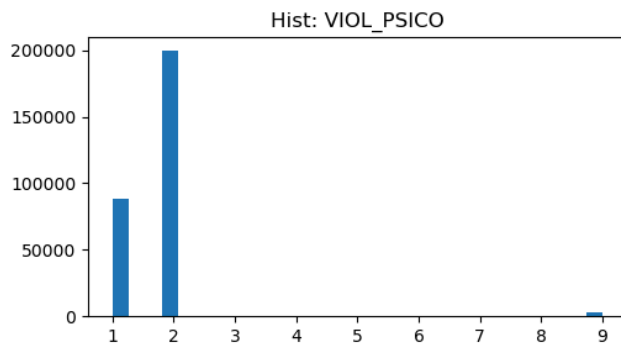
VIOL_MOTIV count 292847.000000
 mean 39.197328
 std 44.030575
 min 1.000000
 25% 1.000000
 50% 9.000000
 75% 88.000000
 max 99.000000
 Name: VIOL_MOTIV, dtype: float64



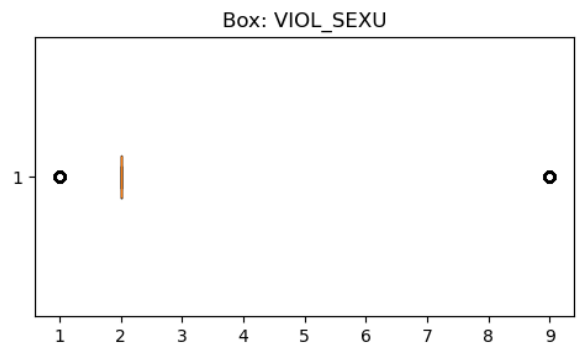
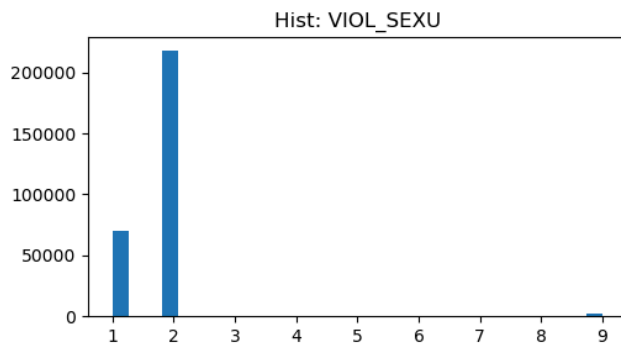
VIOL_NEGLI count 292414.000000
 mean 1.952937
 std 0.774496
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_NEGLI, dtype: float64



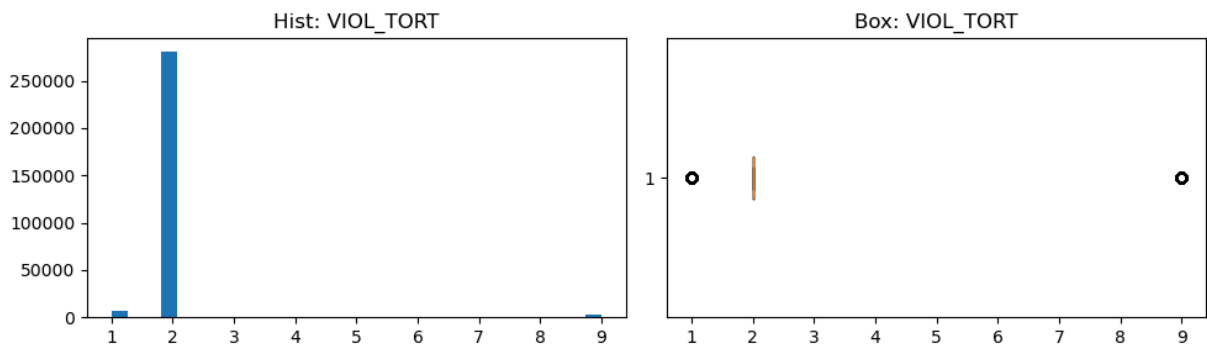
VIOL_OUTR count 290566.000000
 mean 2.095603
 std 0.915889
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_OUTR, dtype: float64



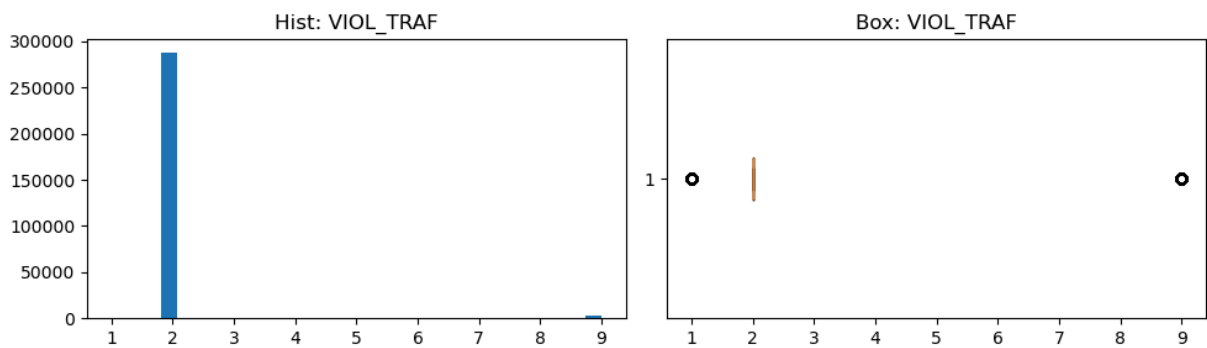
VIOL_PSICO count 290840.000000
 mean 1.762705
 std 0.846780
 min 1.000000
 25% 1.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_PSICO, dtype: float64



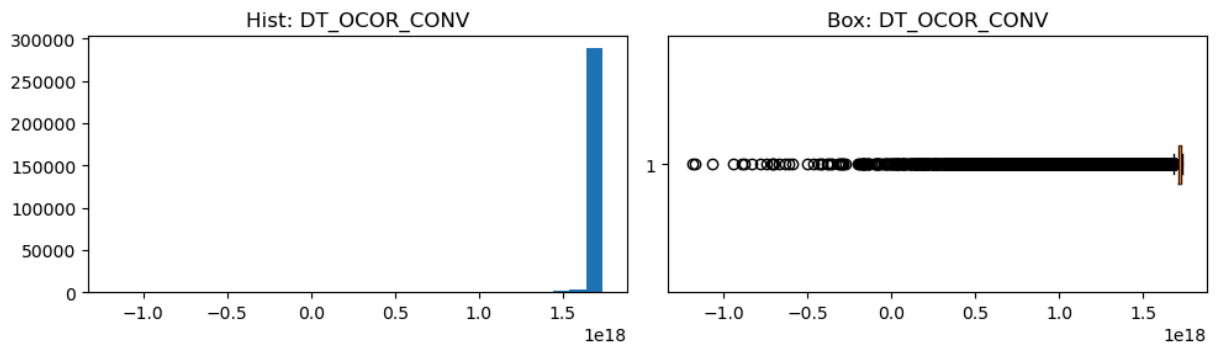
VIOL_SEXU count 291033.000000
 mean 1.821226
 std 0.810266
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_SEXU, dtype: float64



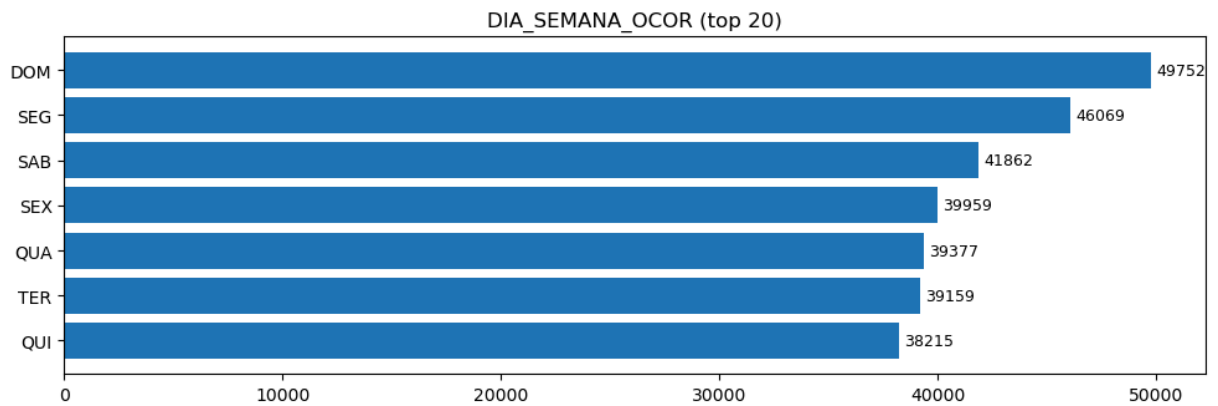
VIOL_TORT count 290668.000000
 mean 2.061138
 std 0.777820
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_TORT, dtype: float64



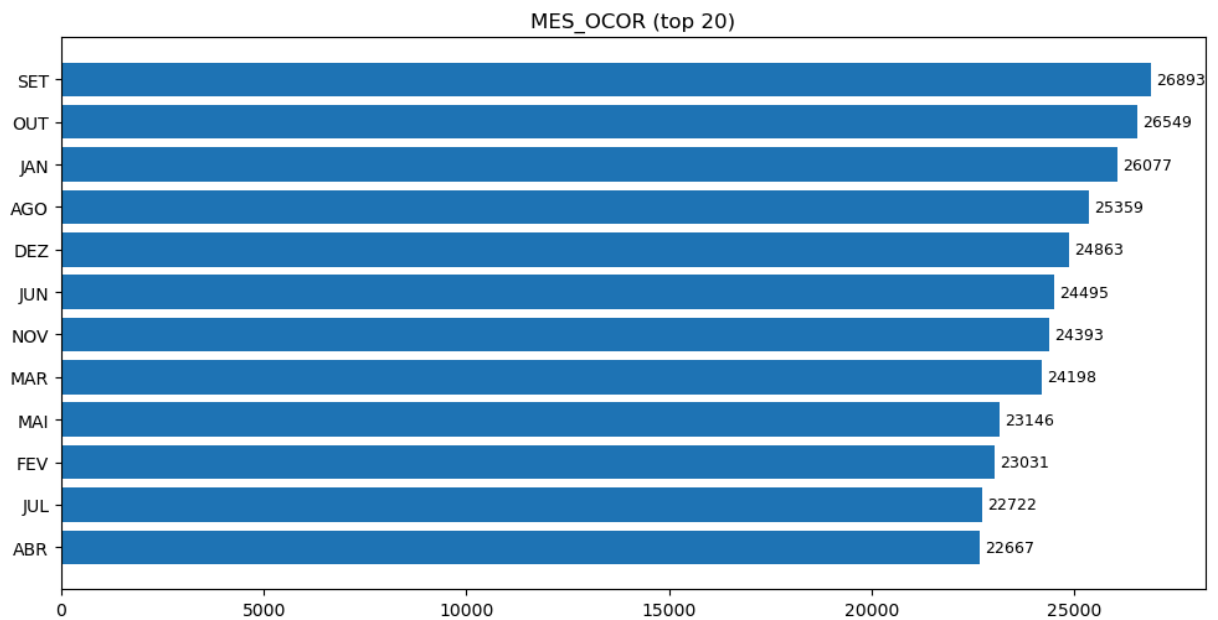
VIOL_TRAF count 290595.000000
 mean 2.068528
 std 0.692712
 min 1.000000
 25% 2.000000
 50% 2.000000
 75% 2.000000
 max 9.000000
 Name: VIOL_TRAF, dtype: float64



```
DT_OCOR_CONV count    2.943930e+05
mean      1.710900e+18
std       7.493195e+16
min      -1.182470e+18
25%      1.709942e+18
50%      1.718582e+18
75%      1.726963e+18
max       1.735603e+18
Name: DT_OCOR_CONV, dtype: float64
```



```
DIA_SEMANA_OCOR count    294393
unique      7
top      DOM
freq      49752
Name: DIA_SEMANA_OCOR, dtype: object
```



```
MES_OC0R count      294393
unique         12
top           SET
freq          26893
Name: MES_OC0R, dtype: object
```